EE 116

FPGA-Based Digital System Design

Lab Tutorial

School of Information Science and Technology

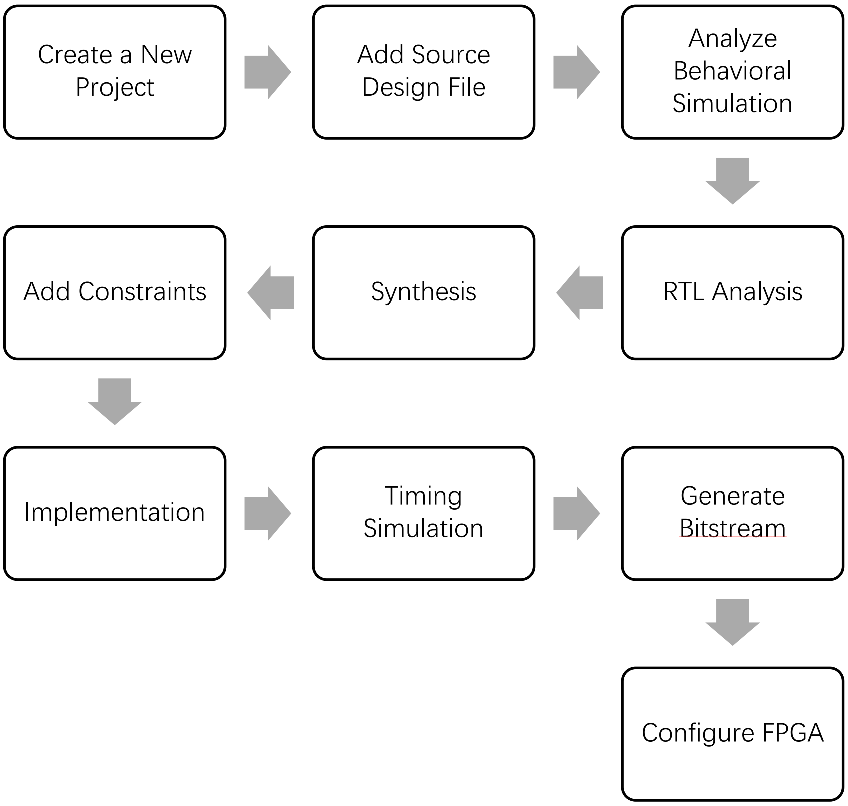
ShanghaiTech University

Sept. 26, 2021

Tutorial 2 Vivado + Zedboard Development Tutorial

Introduction

In our lab we will guide you to create a simple digital design using Xilinx Vivado software. A typical design flow consists of creating new projects, adding source design file, analyzing behavioral simulation, RTL analysis, synthesis, adding constraints, implementation, timing simulation , generating bitstream and configuring FPGA. You will go through the typical design flow targeting the Zynq-7000 All Programmable SoC based ZedBoard. The typical design flow is shown below. We focus on the last seven steps in this tutorial.



A typical design flow

Objective:

Achieve the traditional FPGA development under the environment of Vivado. Here we only consider the PL development.in this experiment, we use switches to control LEDs.

Environment: Vivado 2020.2

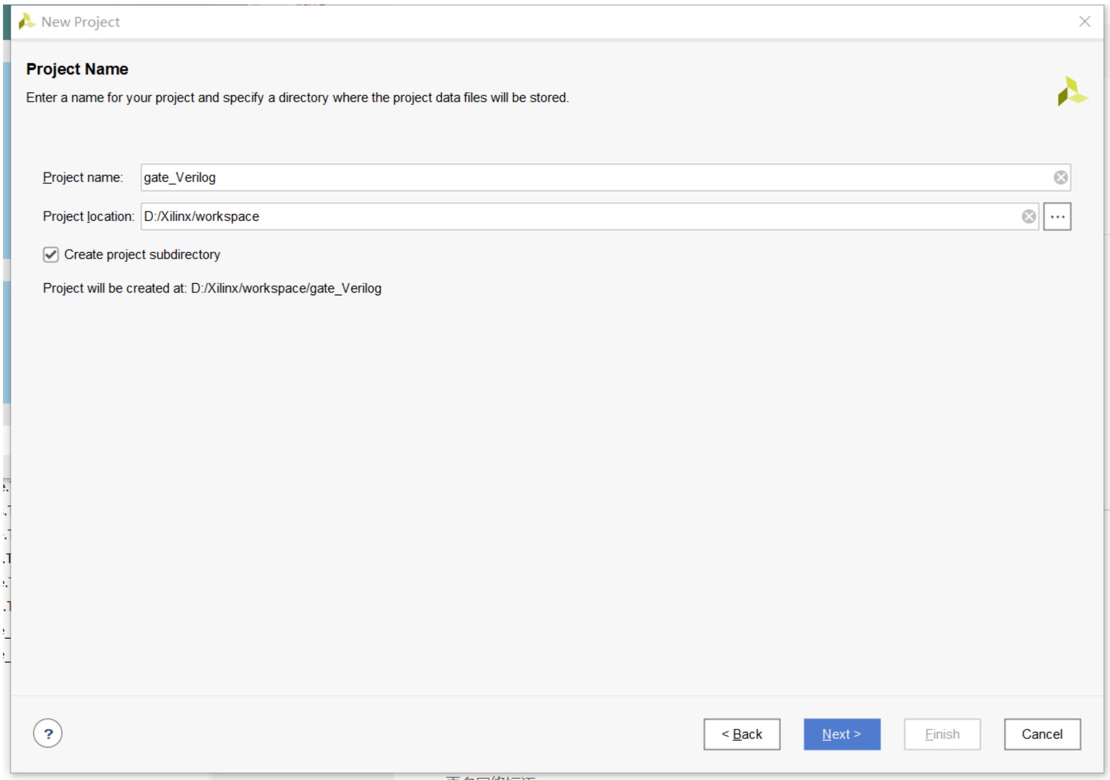
Development board: Zedboard xc7z020clg484-1

Main Flow:

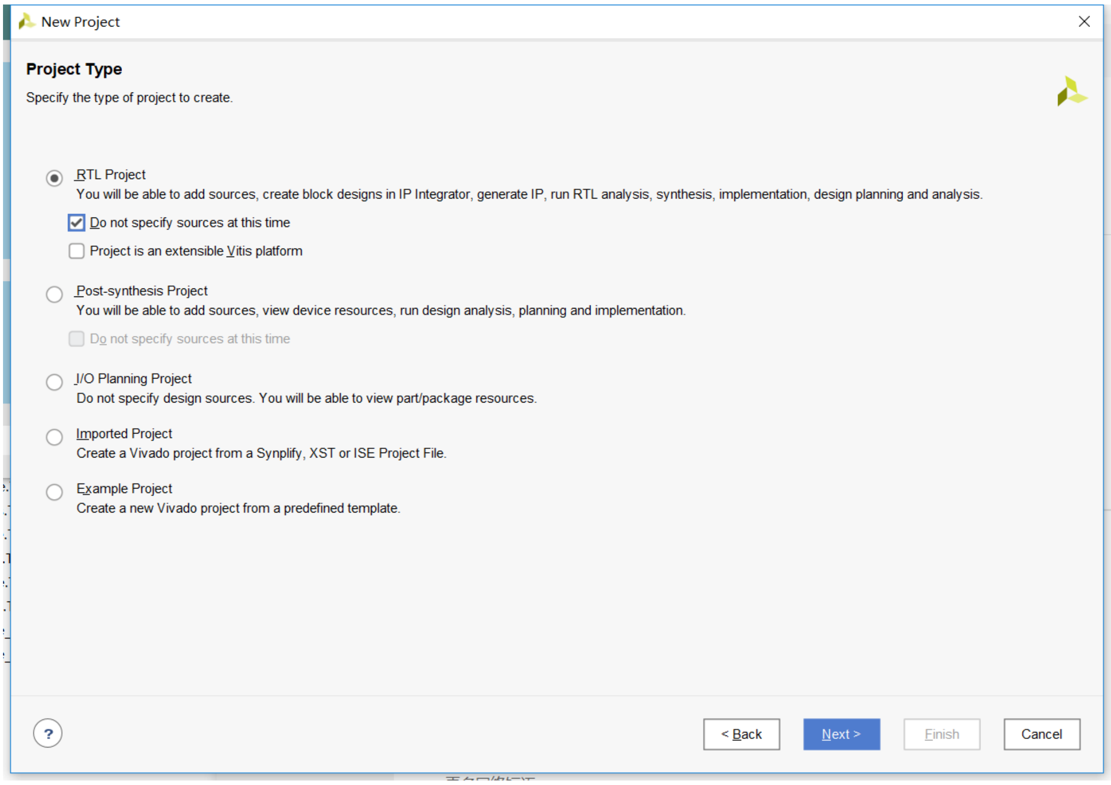
Step1: Create a New project.
Step2: Add Source.
Step3: Behavioral Simulation
Step4: RTL Analysis
Step5: Synthesis
Step6: Add constraints
Step7: Implementation
Step8: Run Post-Implementation Timing Simulation
Step9: Generate Bitstream
Step10: Connect Zedboard and Configure FPGA,

Step1: Create a New project.

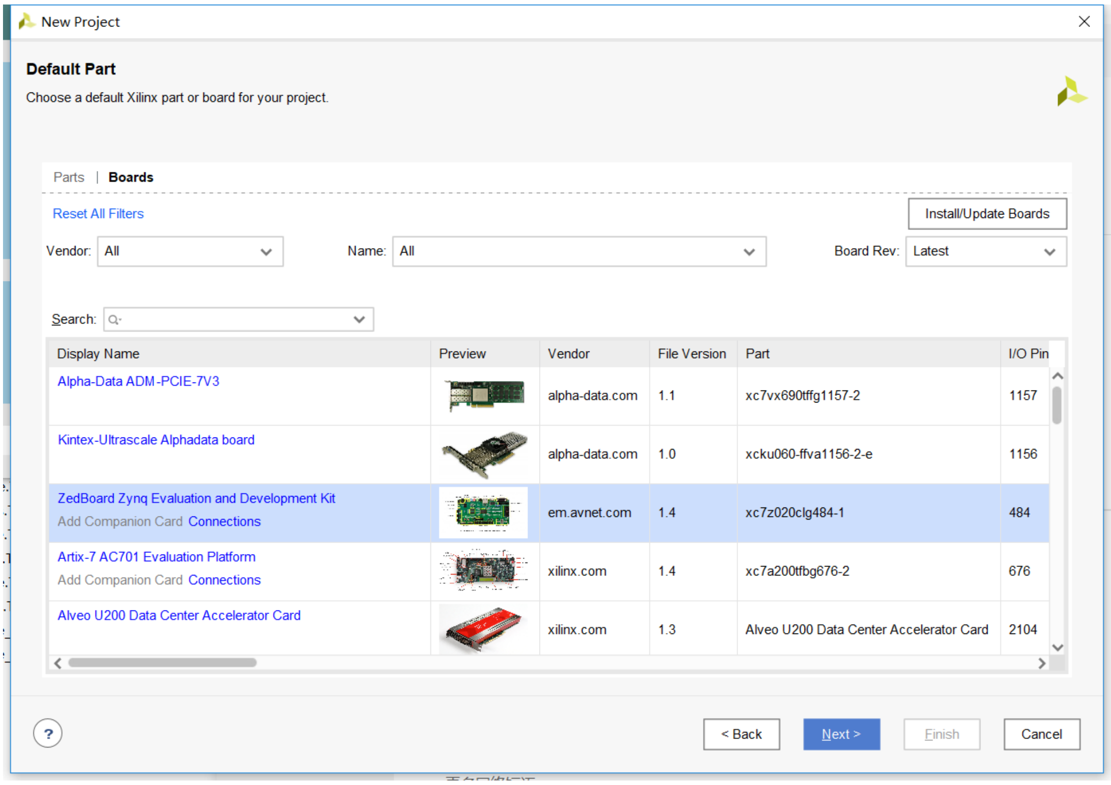
Open Vivado 2020.2.and build a new project., click "**Create a New Vivado Project**", then pop out dialog box "**Project Name**", write your own project name, here we use "**gate_Verilog**", then click "**Next**".

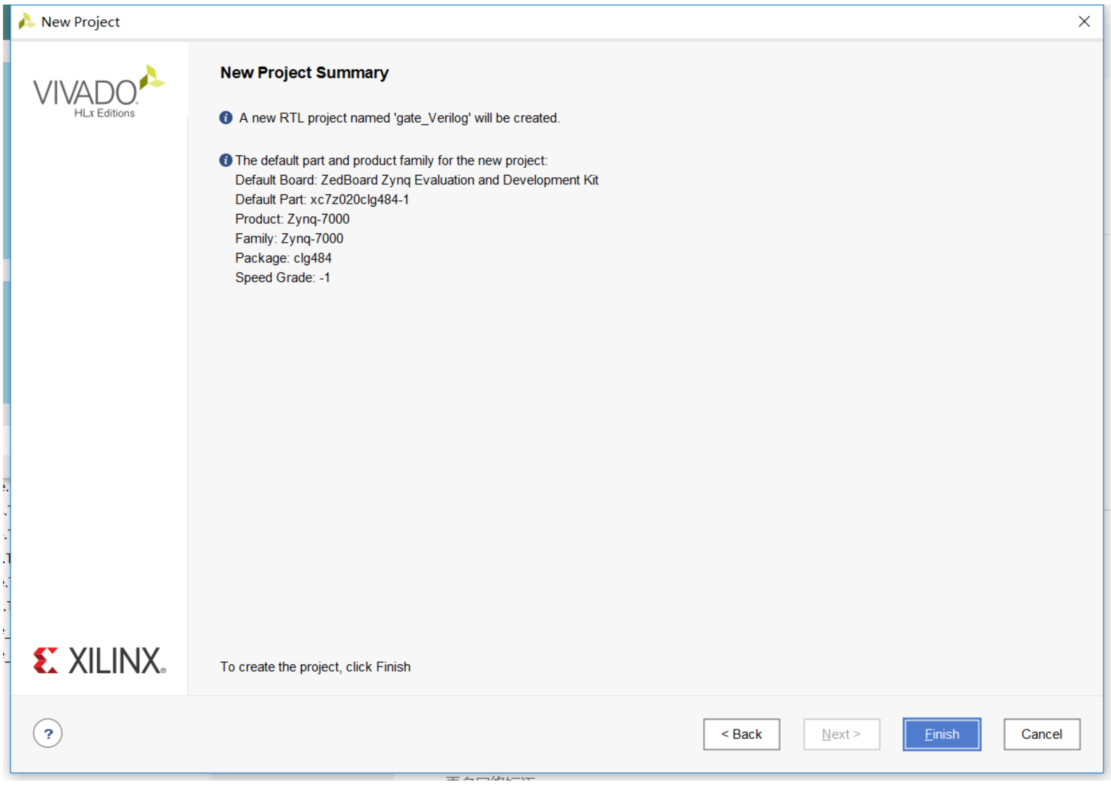


Then we select "**RTL Project**", in the meantime we select "**Do not specify sources at this time**", then click **Next**.



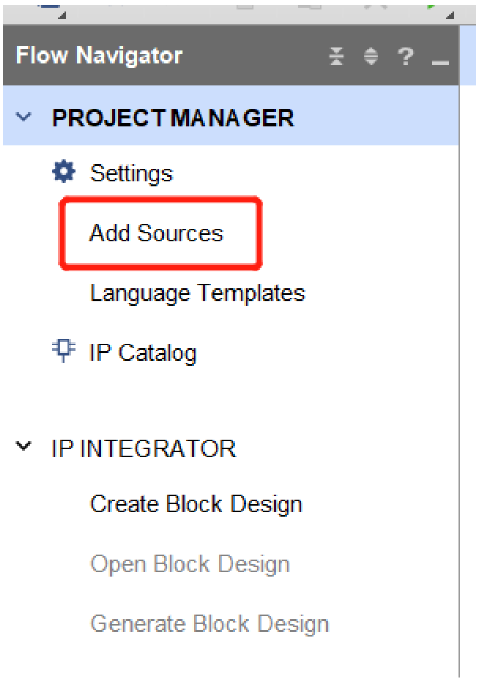
Here click “**Boards**”. and select “**ZedBoard Zynq Evaluation and Development Kit**” board. then click “**Next**”, and click “**Finish**”.



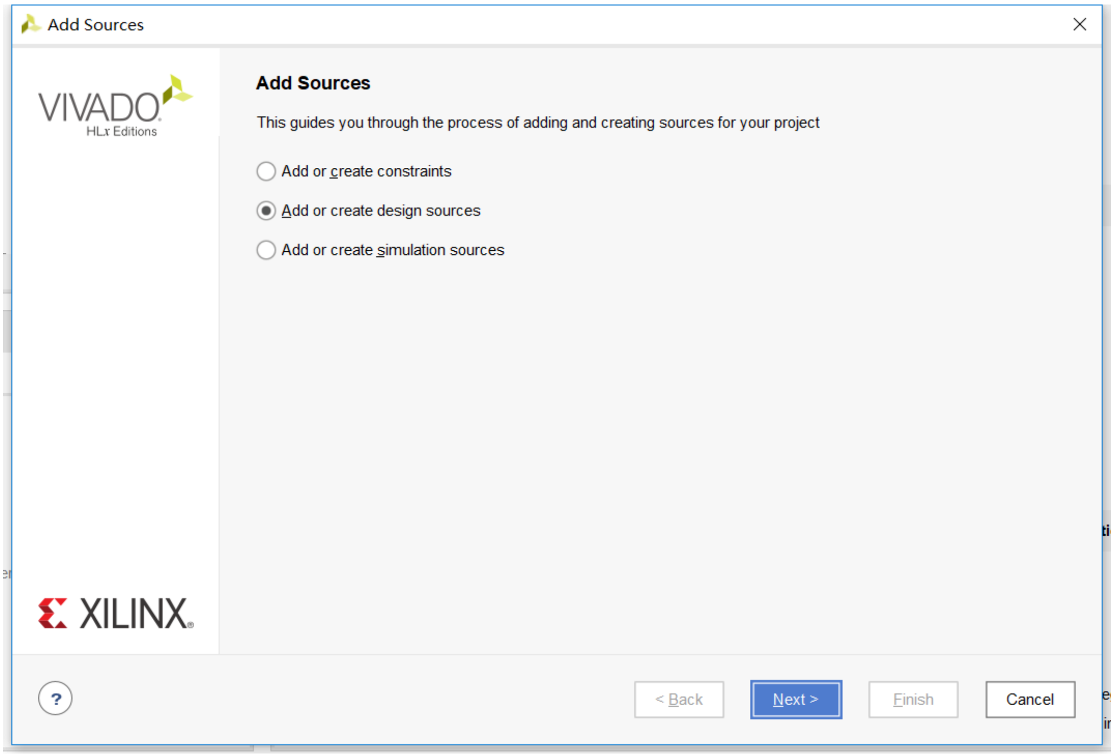


Step2: Add Source.

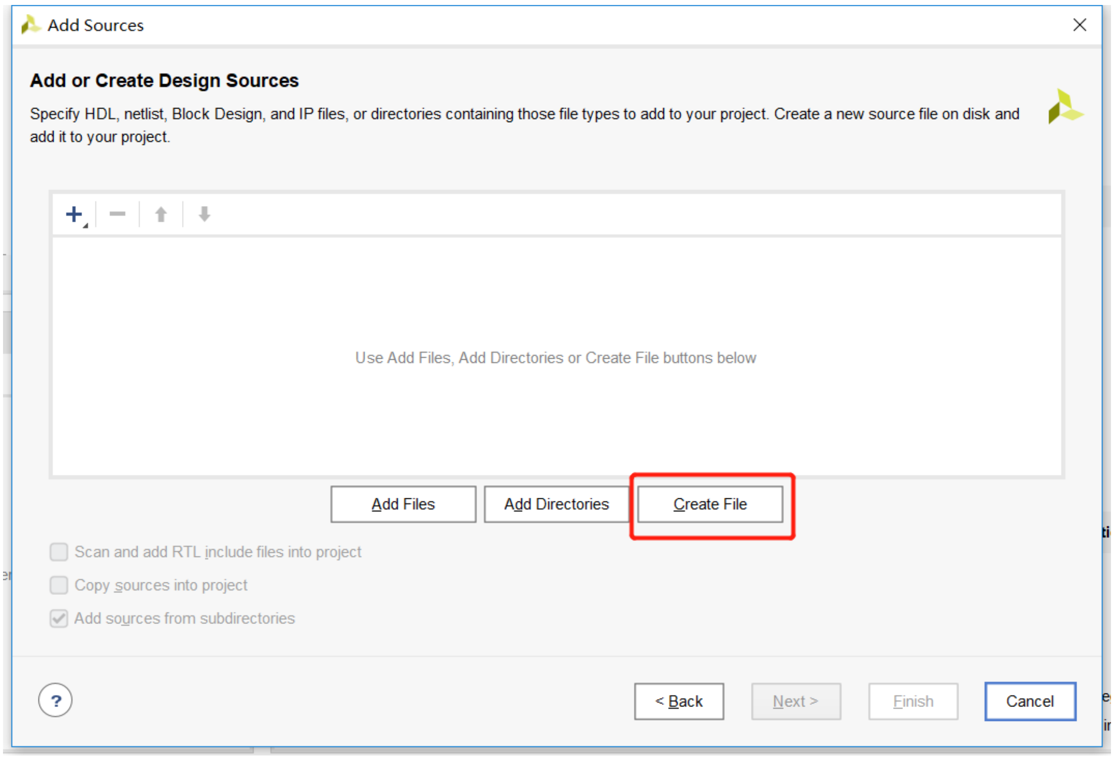
From left window "**Flow Navigator**", Click "**Add Source**",



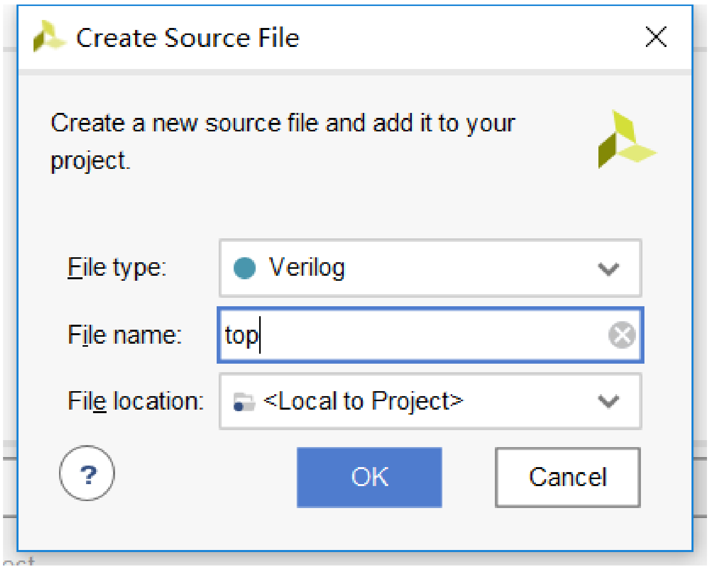
Select "**Add or create design sources**", then Click "**Next**",



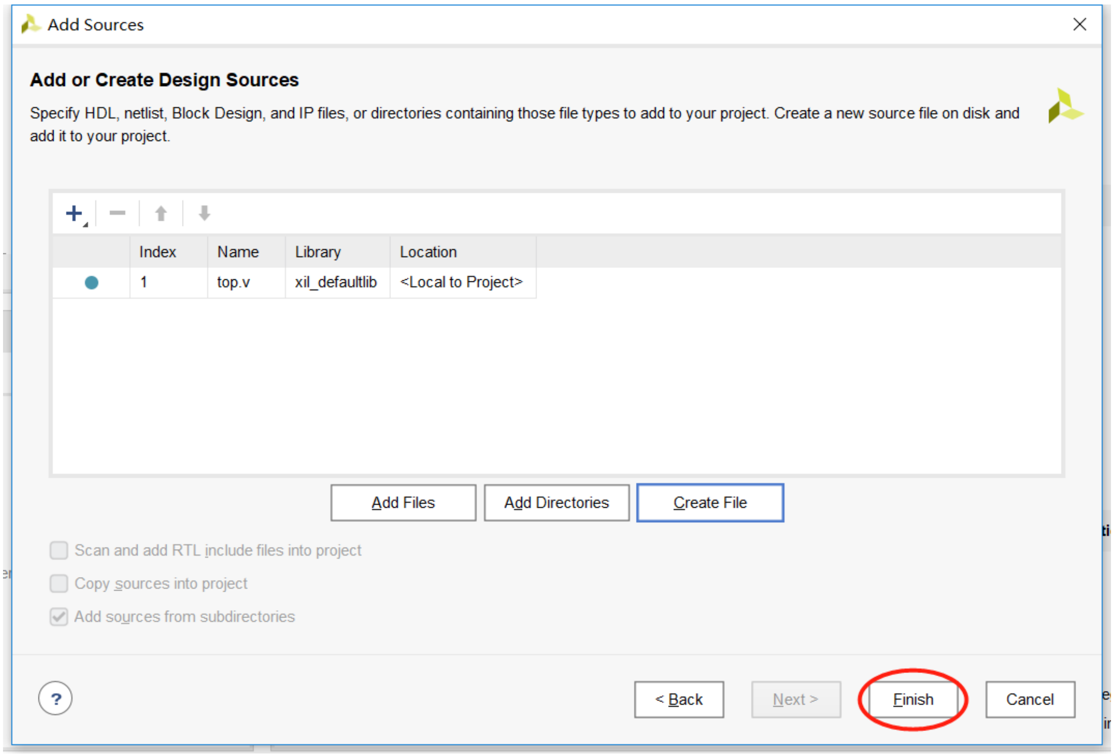
Click “**Create File**”,



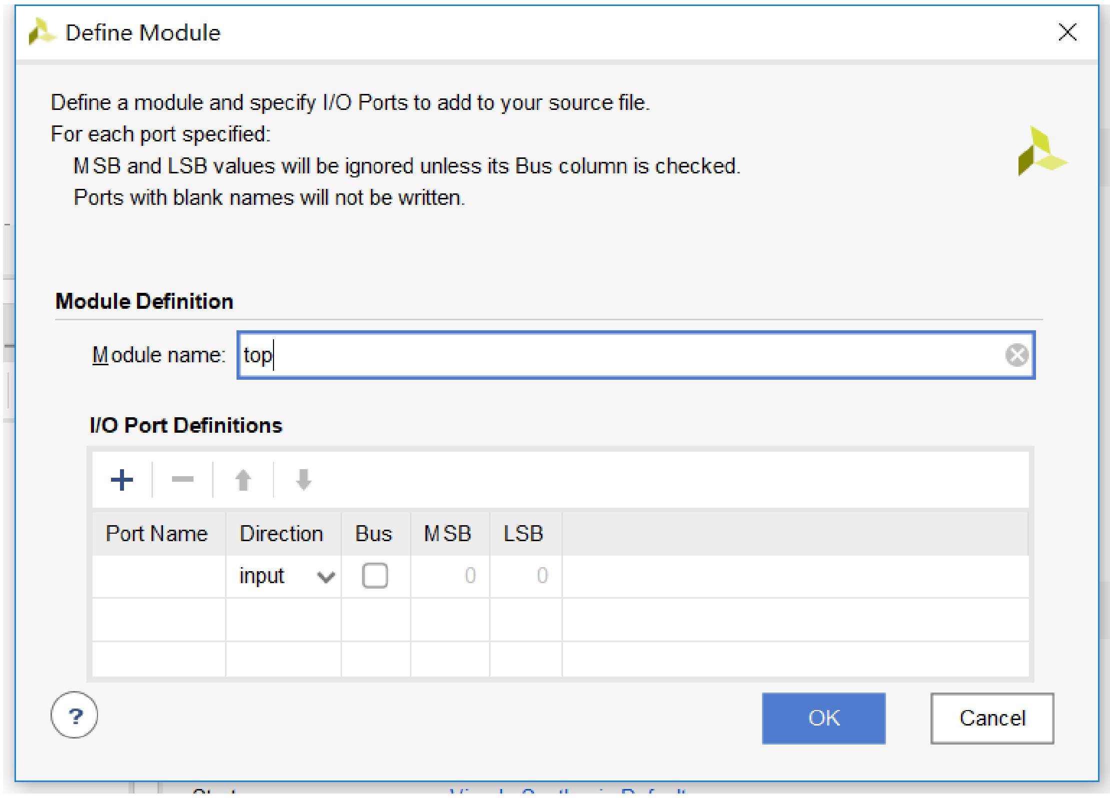
Import the “**File name**”, “**File type**” selects “**Verilog**”. Click “**OK**”,



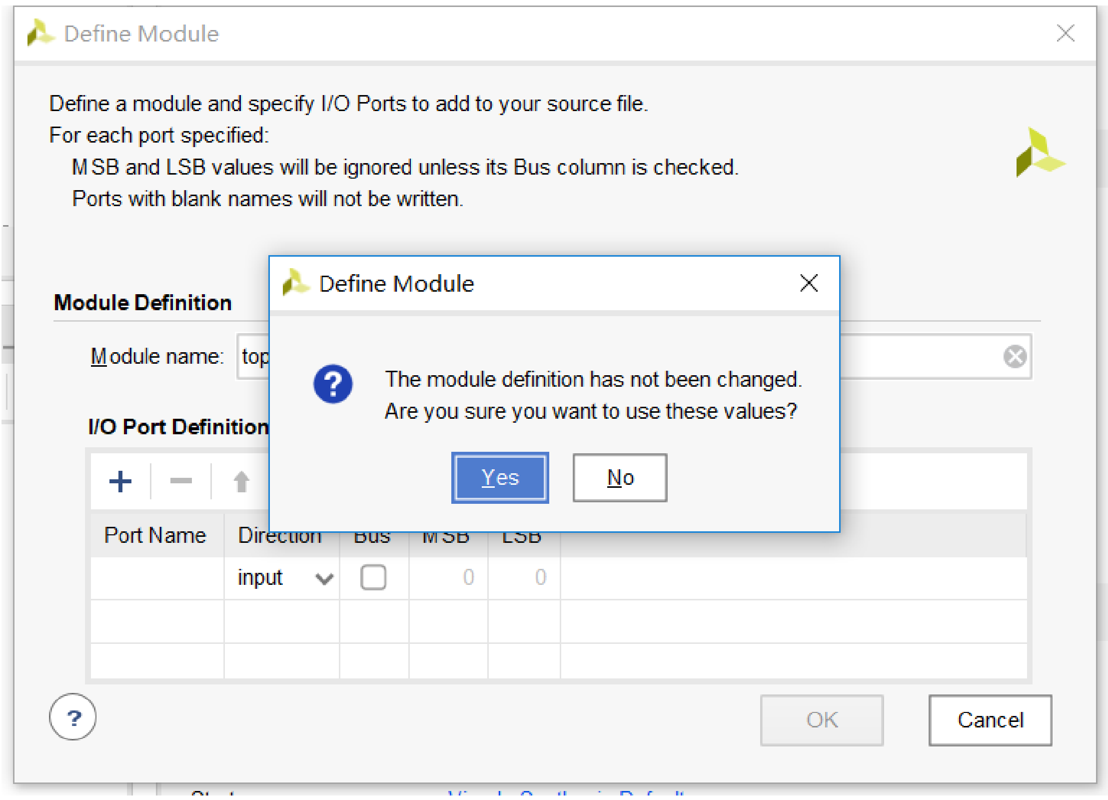
Click “**Finish**”



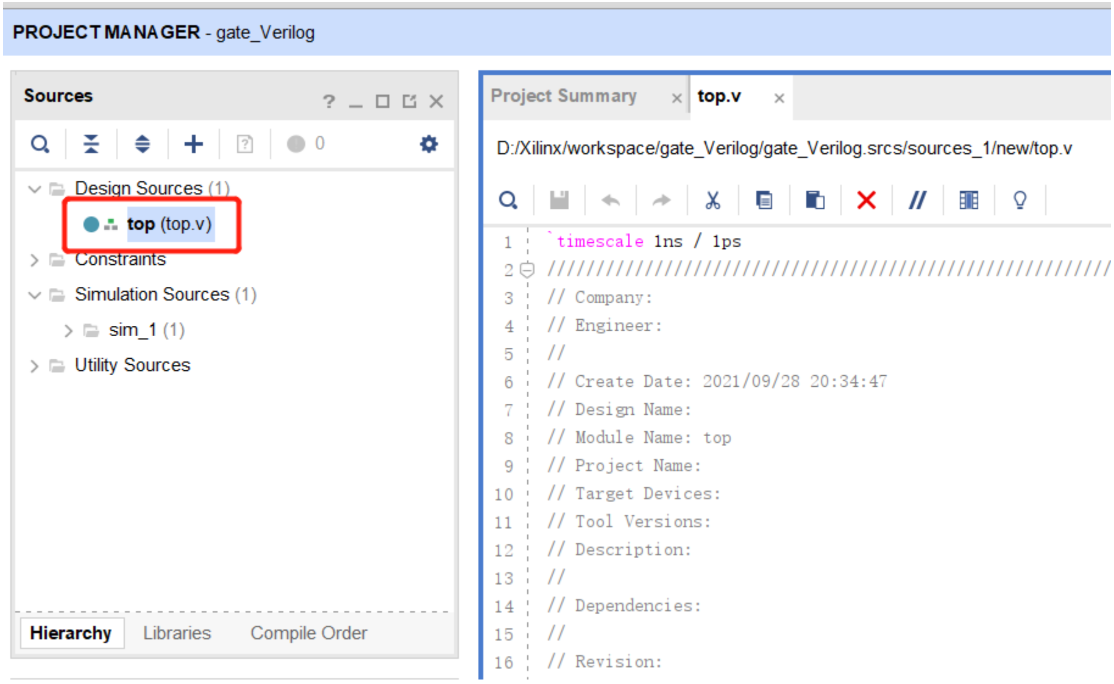
Pop out the dialog box “**Define Module**”, Click “**OK**”,



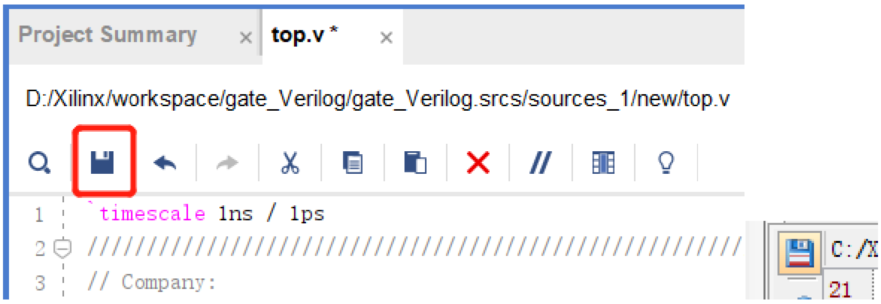
Click “**Yes**”



Find Source Window. Double click “**top (top.v)**”. Open the edit box.

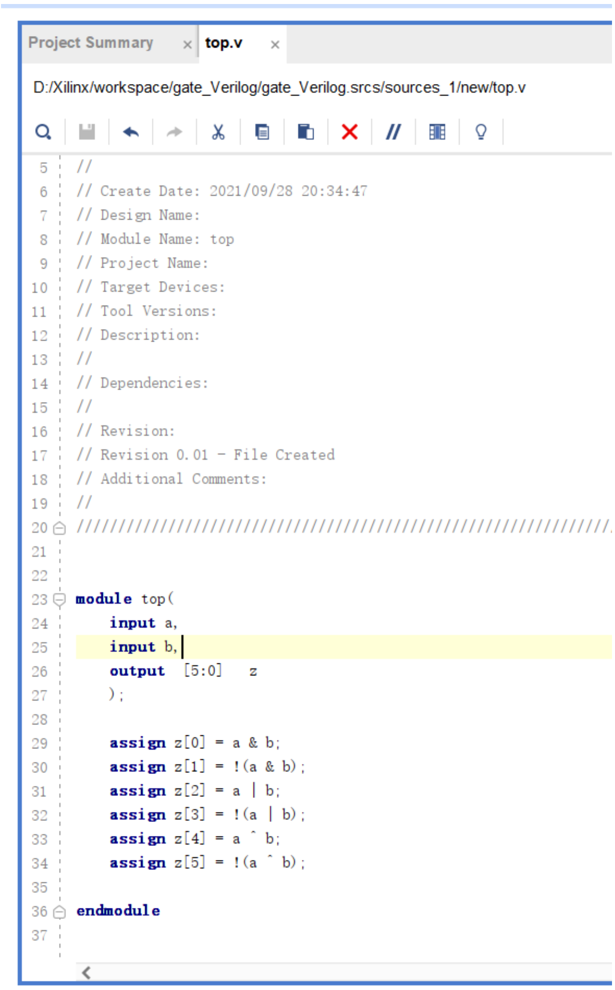


Write Verilog code, Click "**save**"



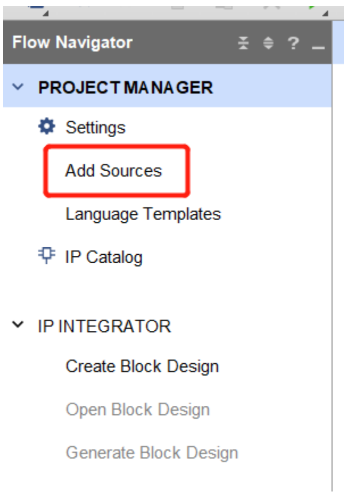
top.v

```
module top(
    input a,
    input b,
    output   [5:0]    z
    );
    assign z[0] = a & b;
    assign z[1] = !(a & b);
    assign z[2] = a | b;
    assign z[3] = !(a | b);
    assign z[4] = a ^ b;
    assign z[5] = !(a ^ b);

endmodule
```

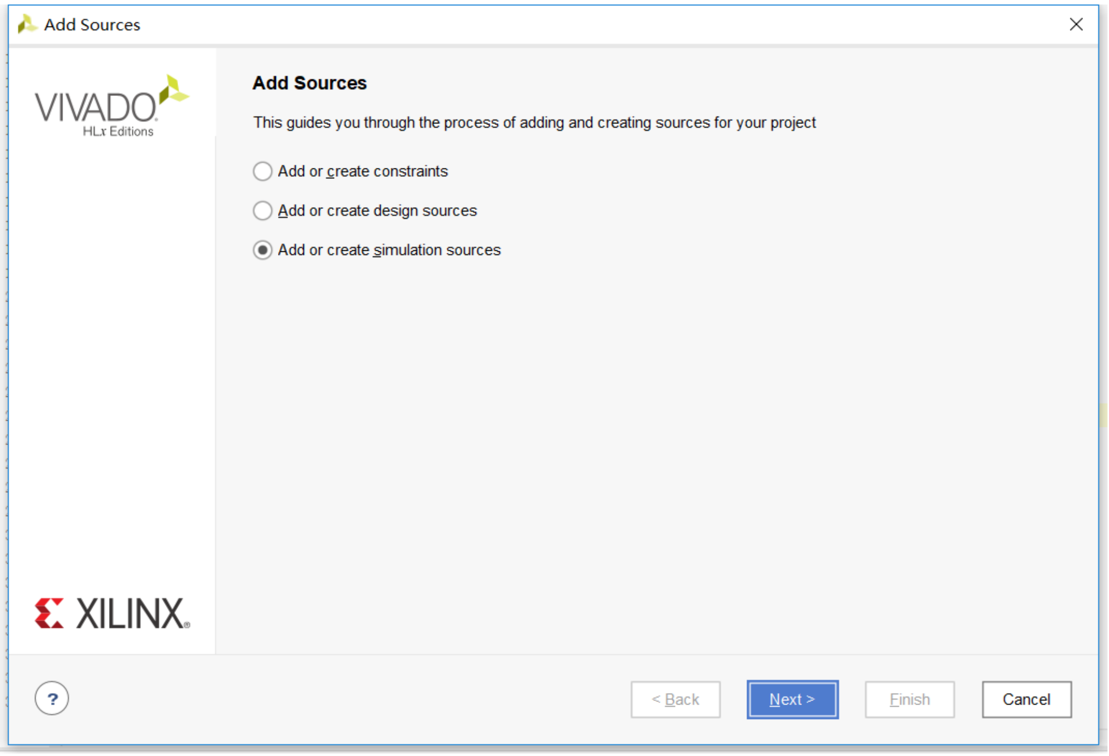


Step3: Behavioral Simulation

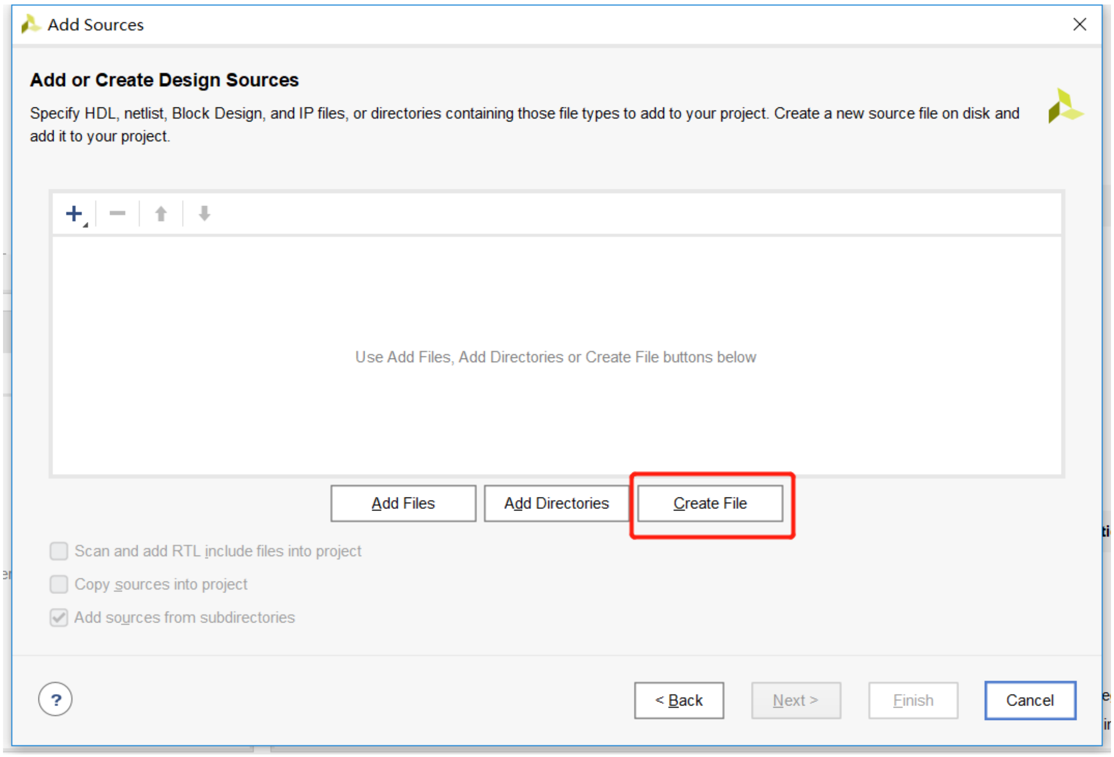
Click “**Add Sources**”



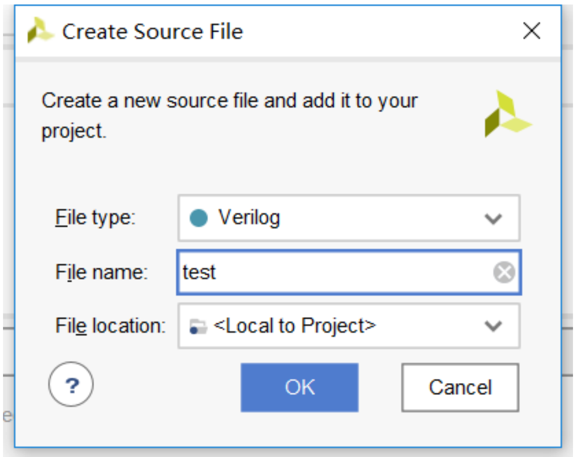
Select “**Add or create simulation sources**”,Click “**Next**”



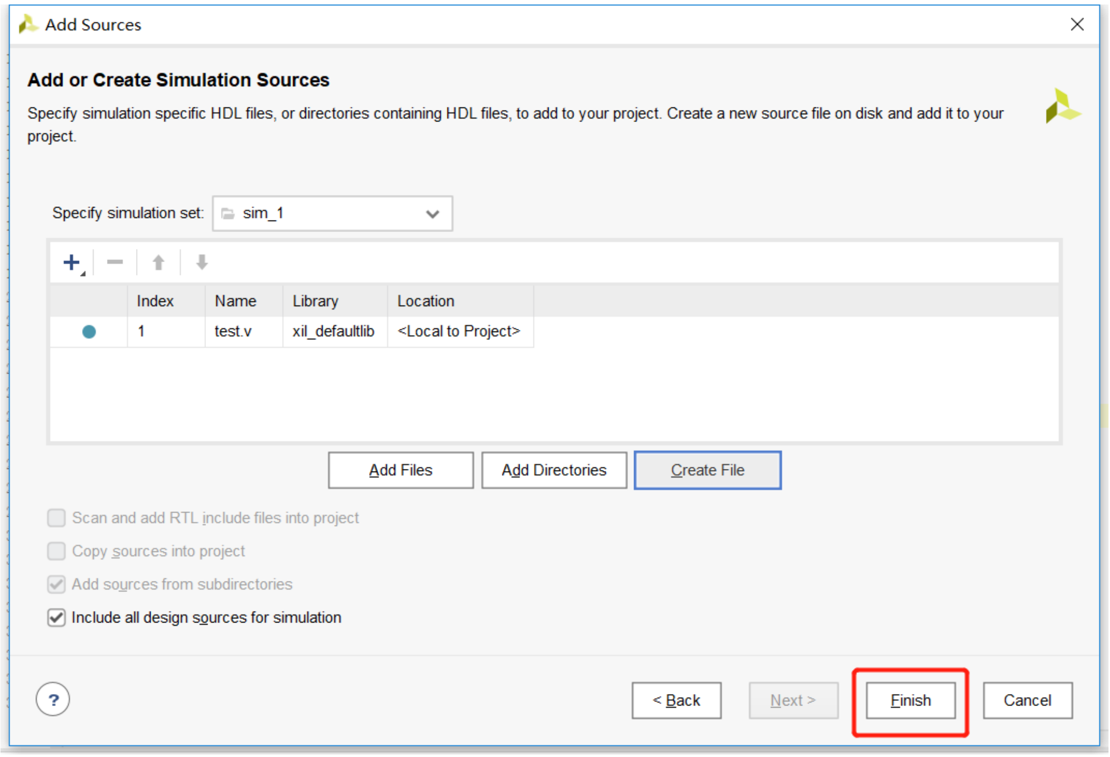
Click “**Create File**”



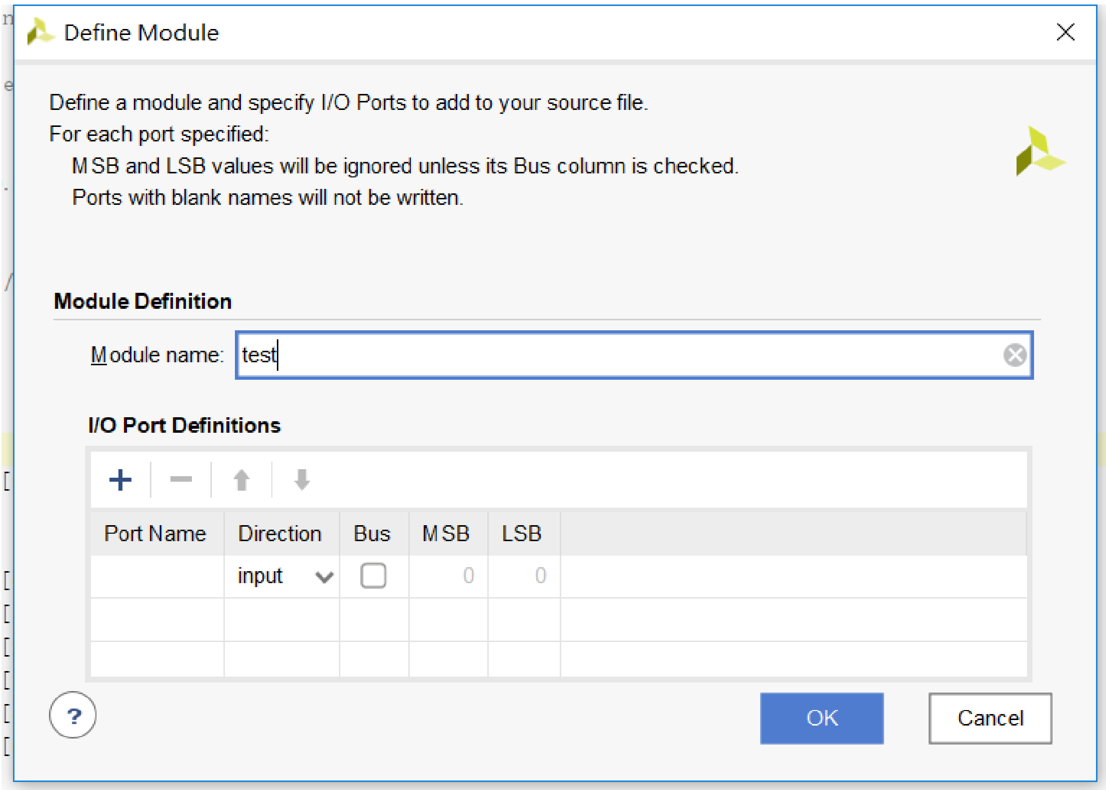
Write the File name, and click “**OK**”, File type selects “**Verilog**”



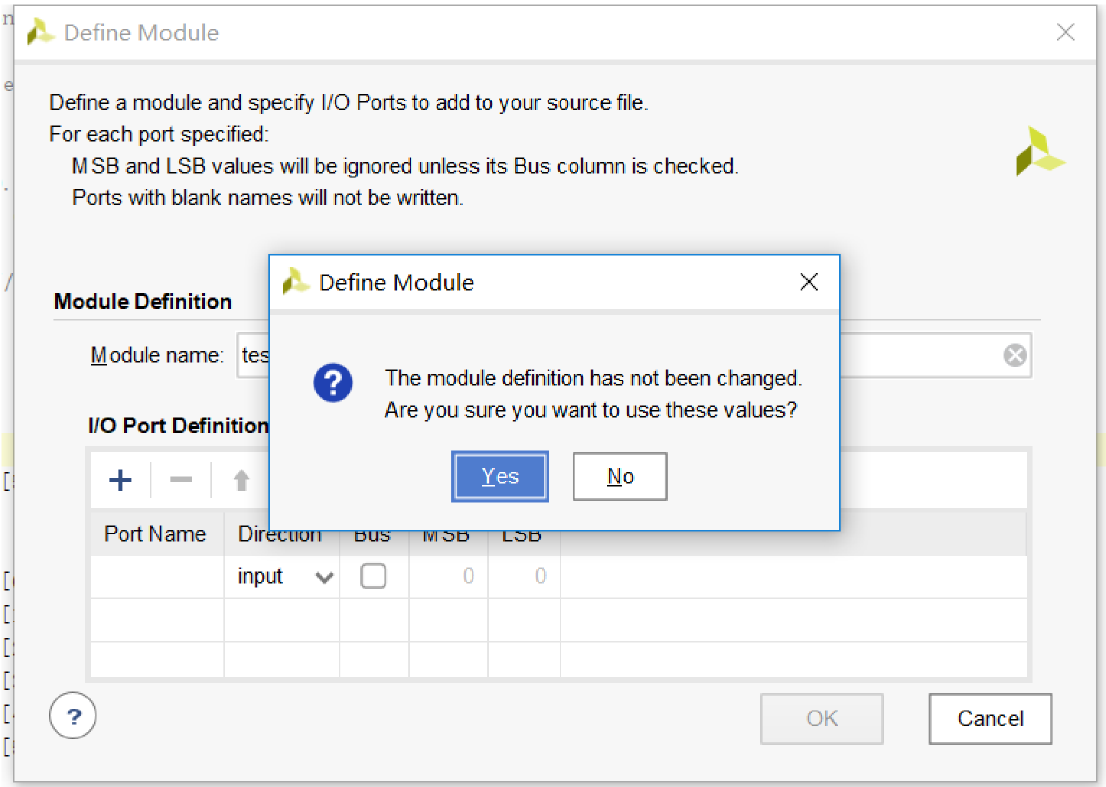
Click “**Finish**”



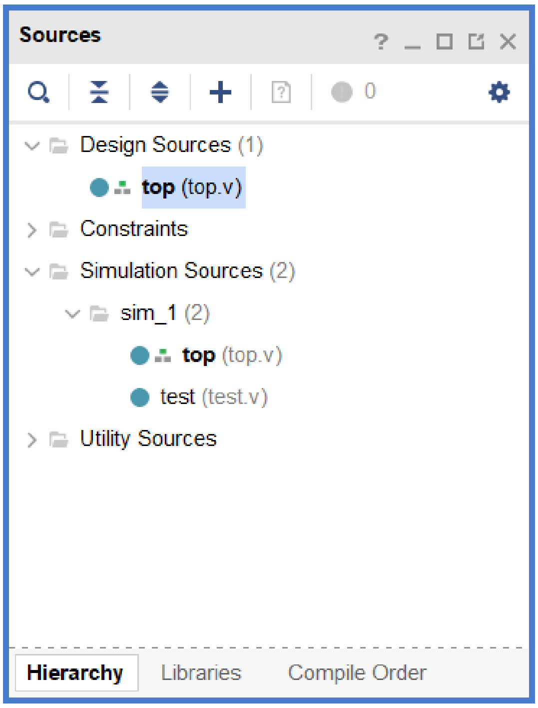
Click “**OK**”



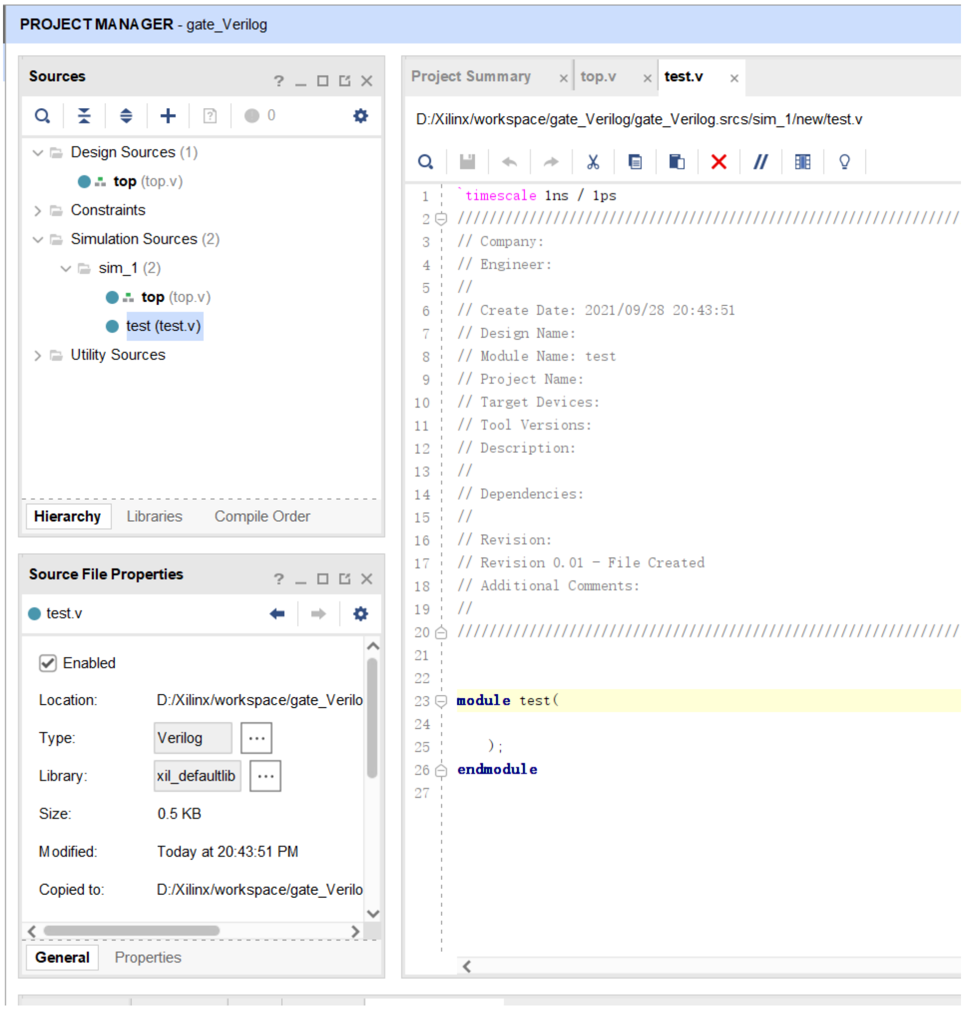
Click “**Yes**”



We have added test file under “**Simulation Sources**”



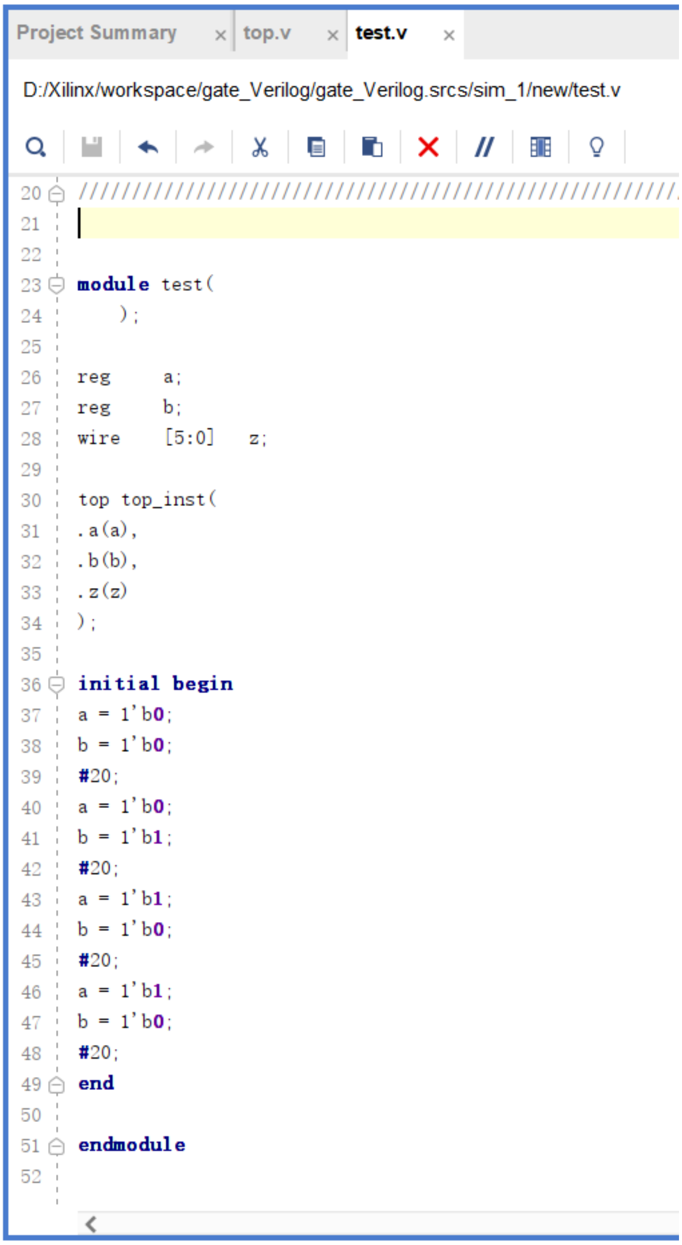
Open “**test.v**” file.



Then add test code as follows:

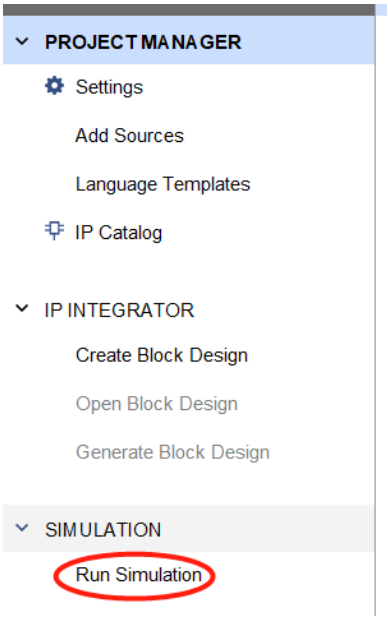
test.v

```
`timescale 1ns/1ps
module test(
        );

reg     a;
reg     b;
wire    [5:0]   z;

top top_inst(
.a(a),
.b(b),
.z(z)
);

initial begin
a = 1b0;
b = 1b0;
#20;
a = 1b0;
b = 1b1;
#20;
a = 1b1;
b = 1b0;
#20;
a = 1b1;
b = 1b 1;
#20;
end

endmodule
```

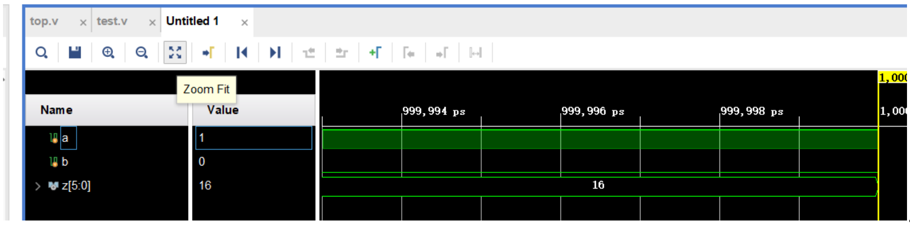


Click **save**

Click "**Run Simulation**", select "**Run Behavioral Simulation**"



Click Zoom fit

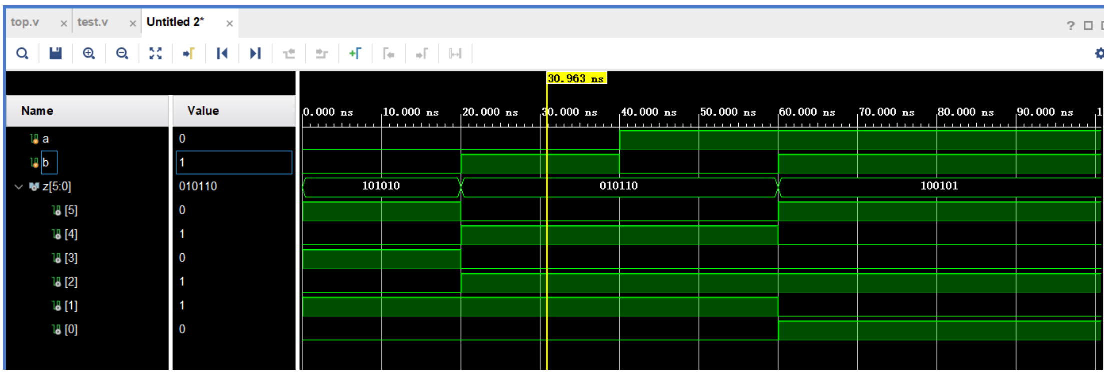


,

right click Value of z[5:0],click “**Radix**”, click “**Binary**”, check results with truth table.

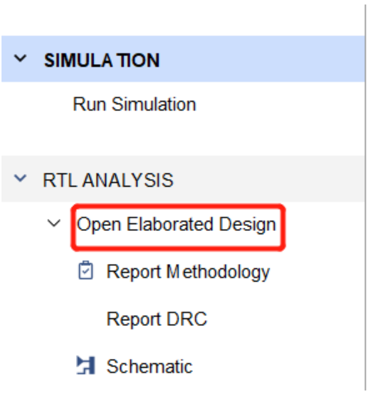
Truth Table

a	b	Z[5:0]
0	0	101010
0	1	010110
1	0	010110
1	1	100101

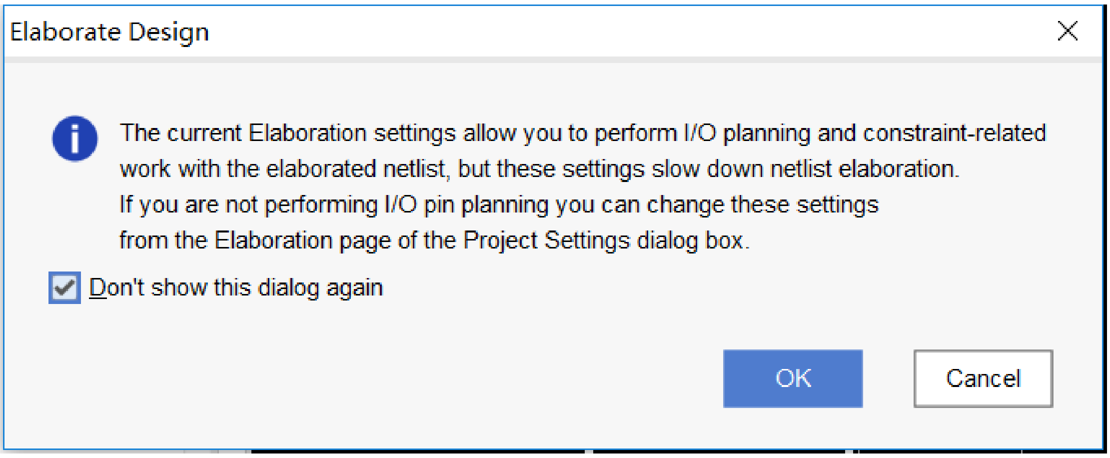


Step4: RTL Analysis

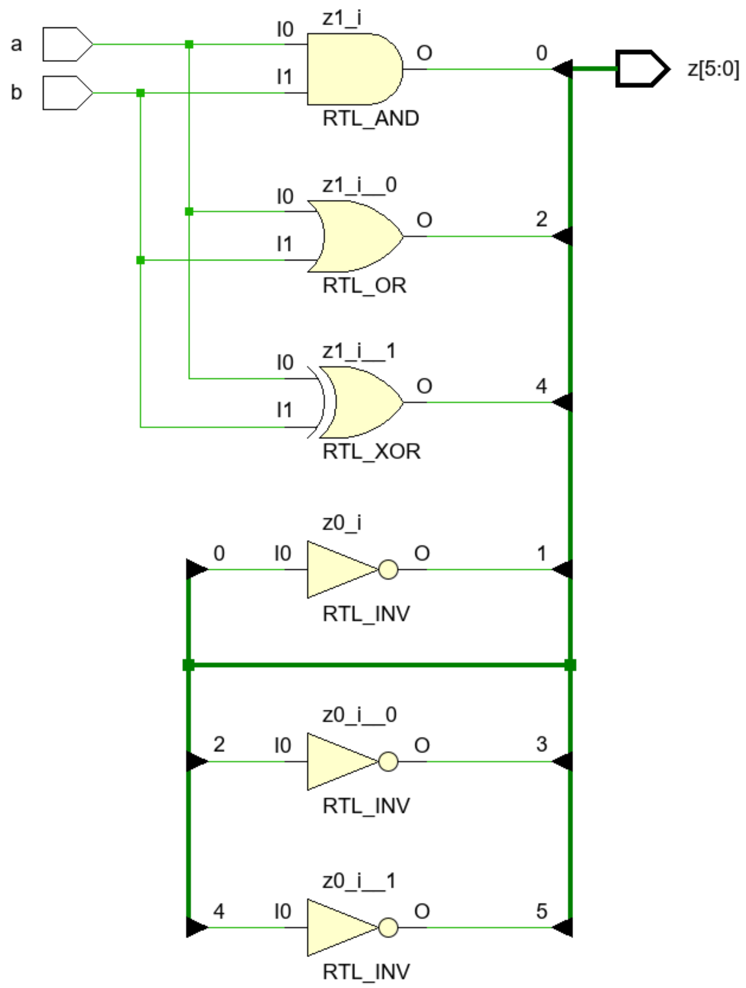
Click “**Open Elaborated Design**”



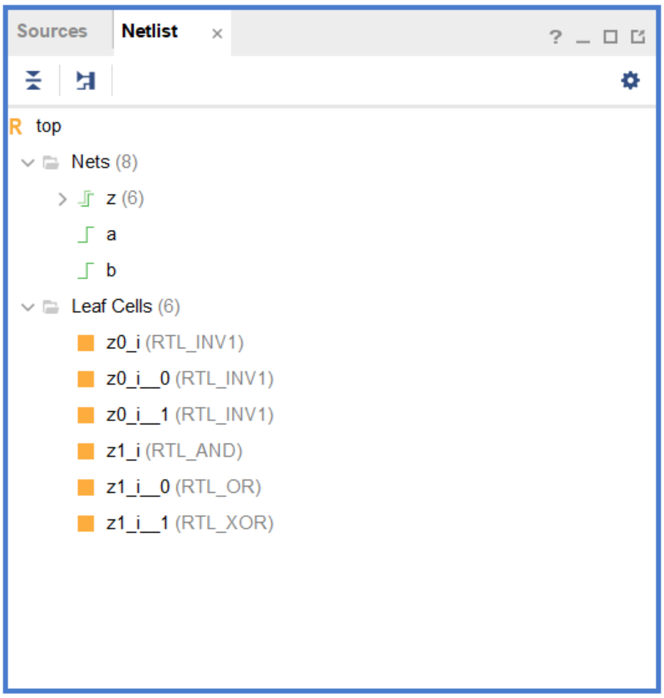
Click “**OK**”



Click “**Schematic**” after RTL process.

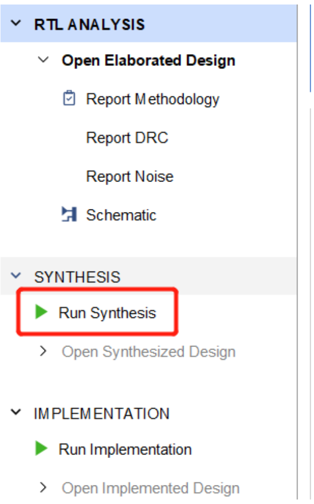


View “**RTL Netlist**”

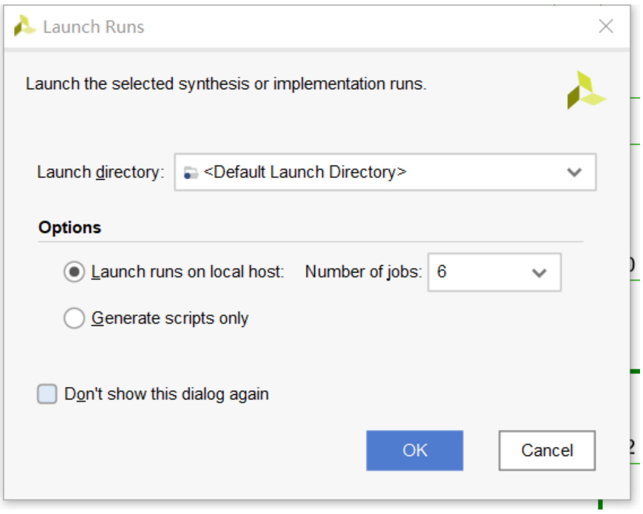


Step5: Synthesis

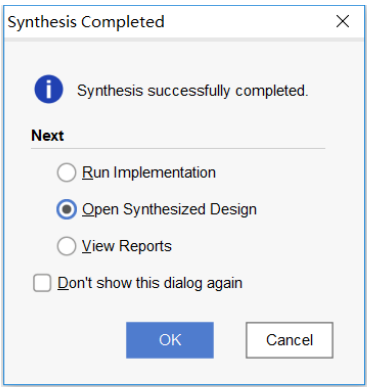
Click "**Run Synthesis**"



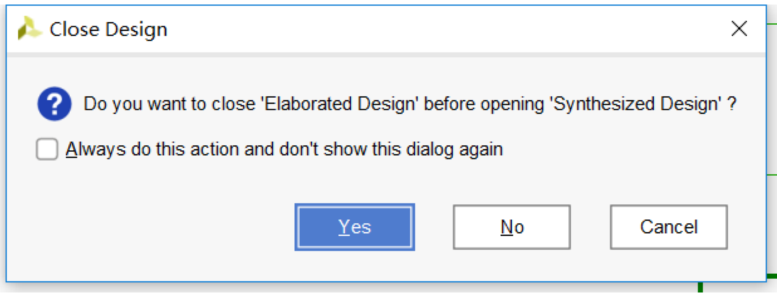
Click OK, to do synthesis.



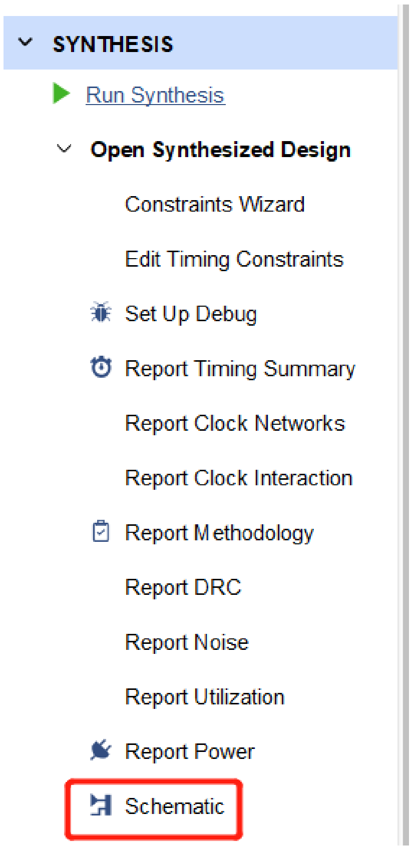
After Synthesis Completed, Select "**Open Synthesized Design**", Click "**OK**"

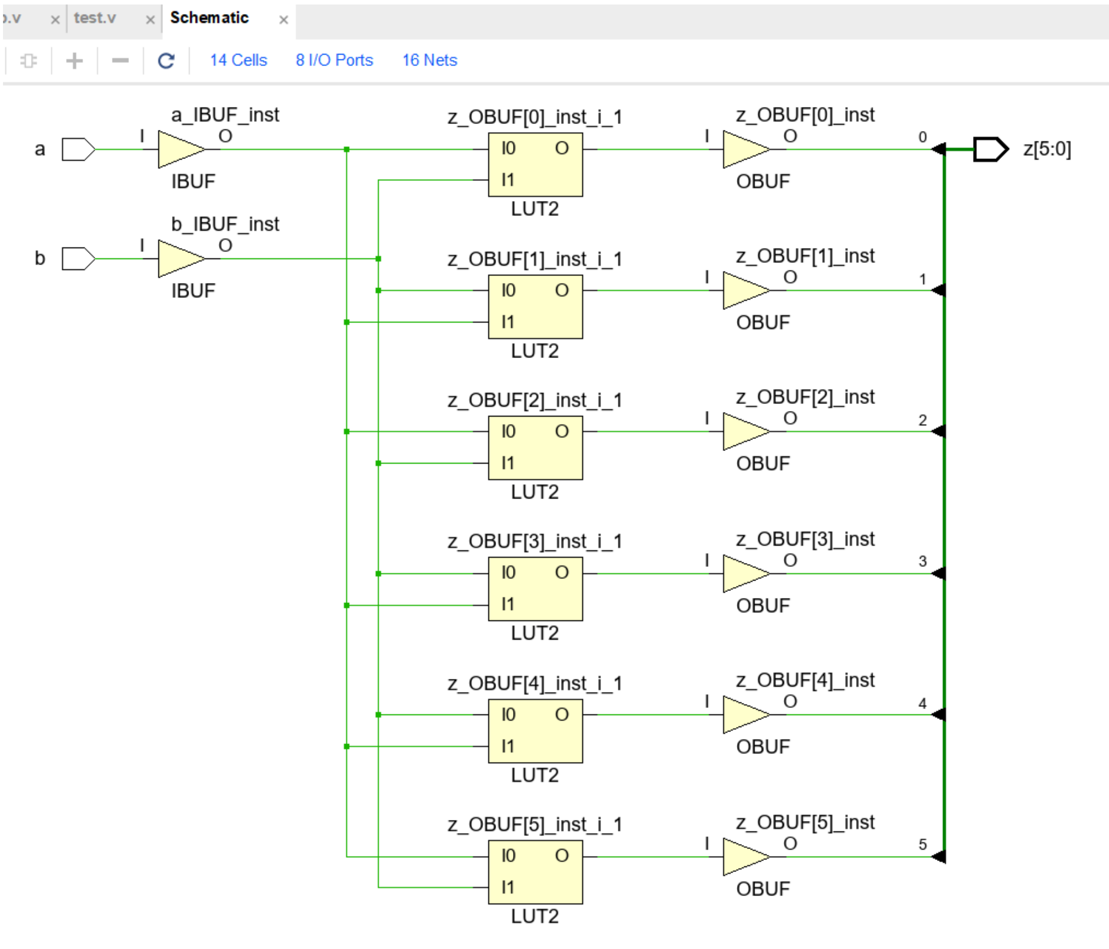


Click "**Yes**"

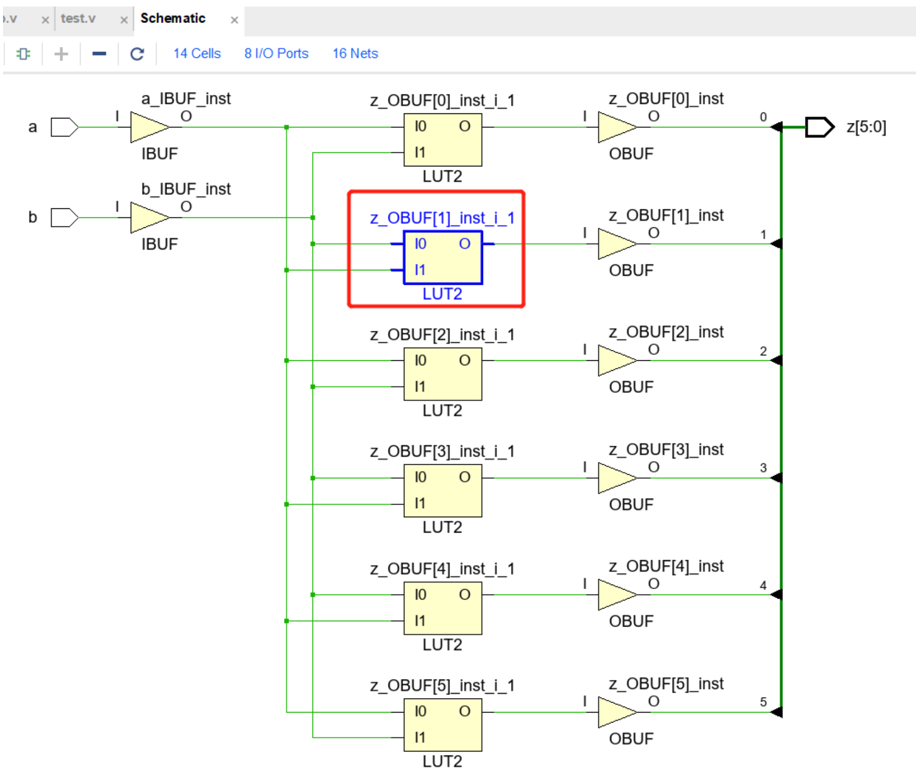


Click “**Schematic**”, and there are also many reports

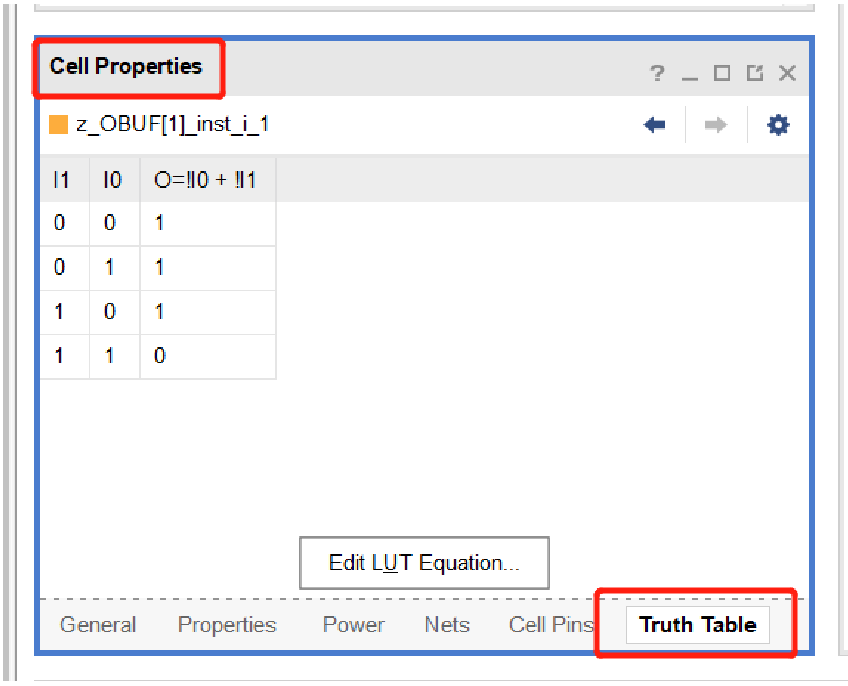




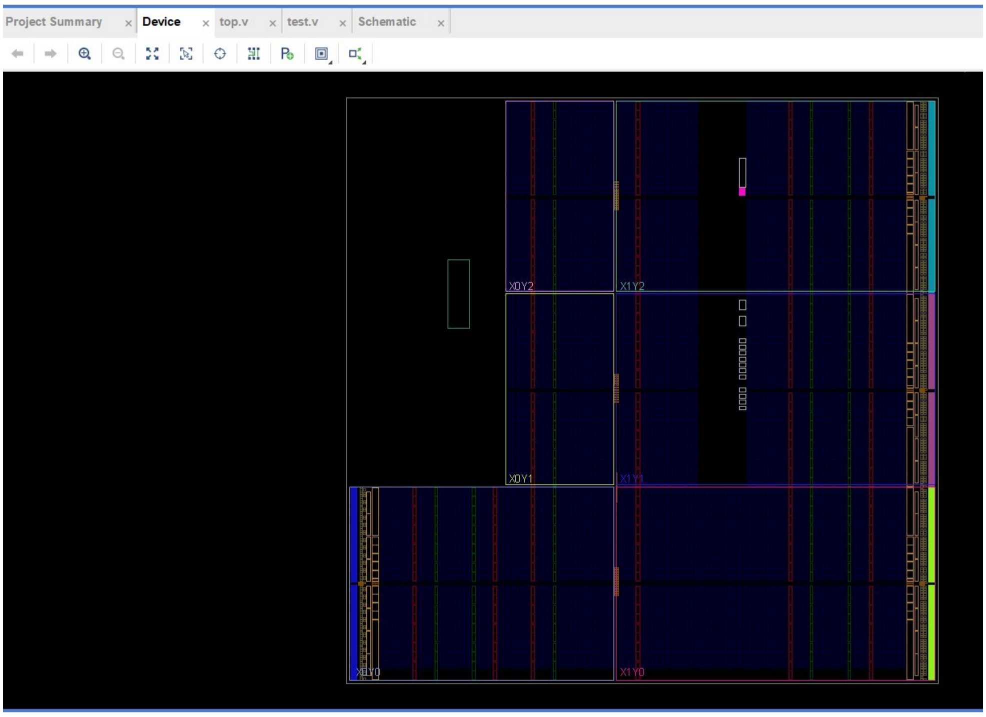
Double click one of LUTs.



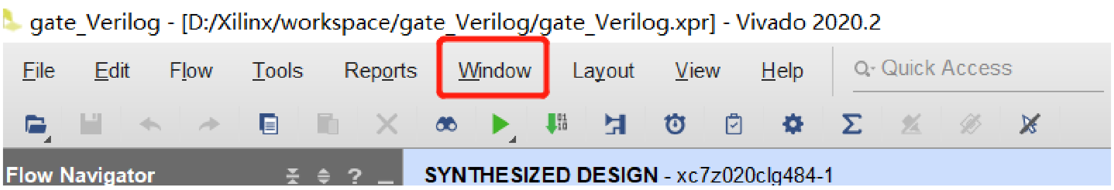
Find Cell Properties ,Click “**Truth Table**”, we can check “**Truth Table**”

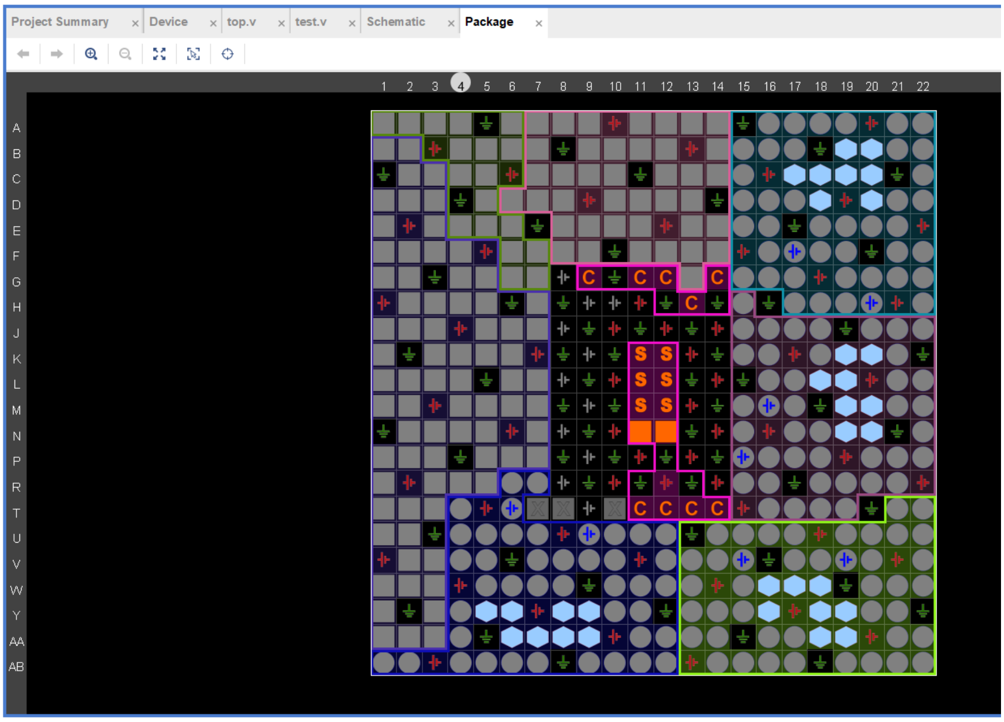


Also we can check “**Device**”



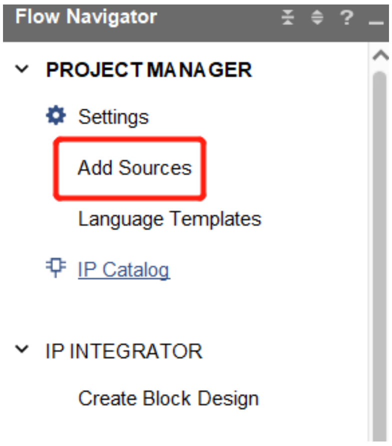
Also. we can click “**window**”, select “**Package**”



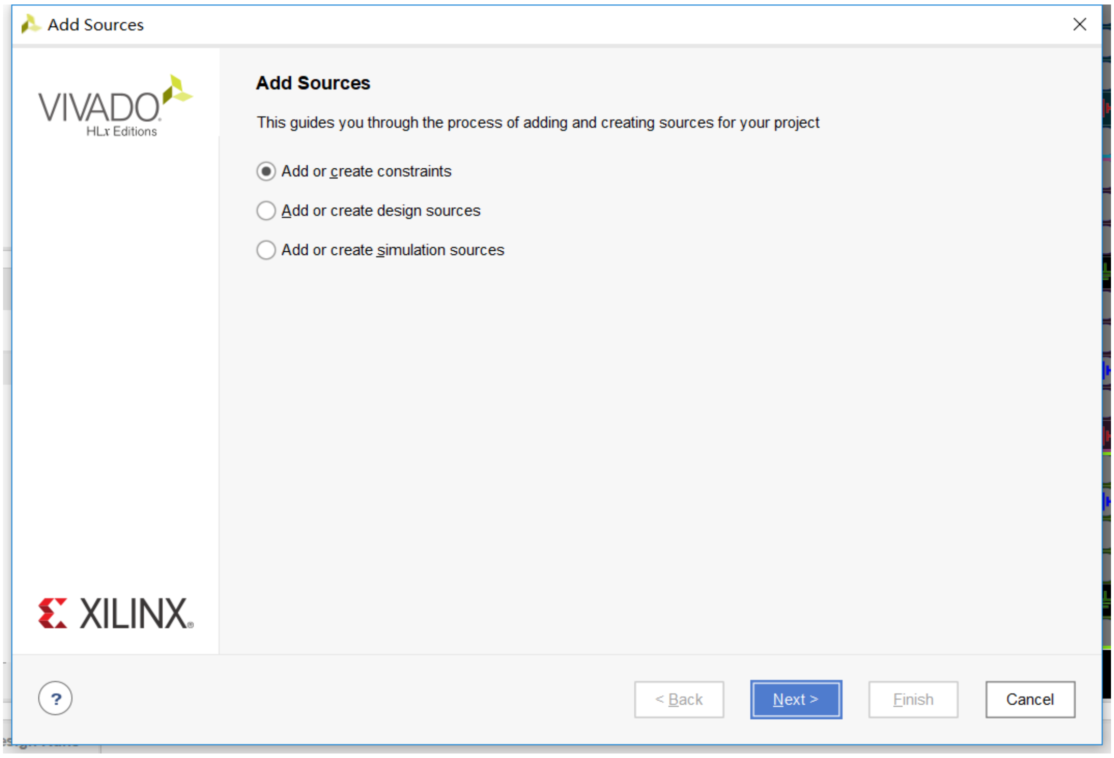


Step6: Add constraints

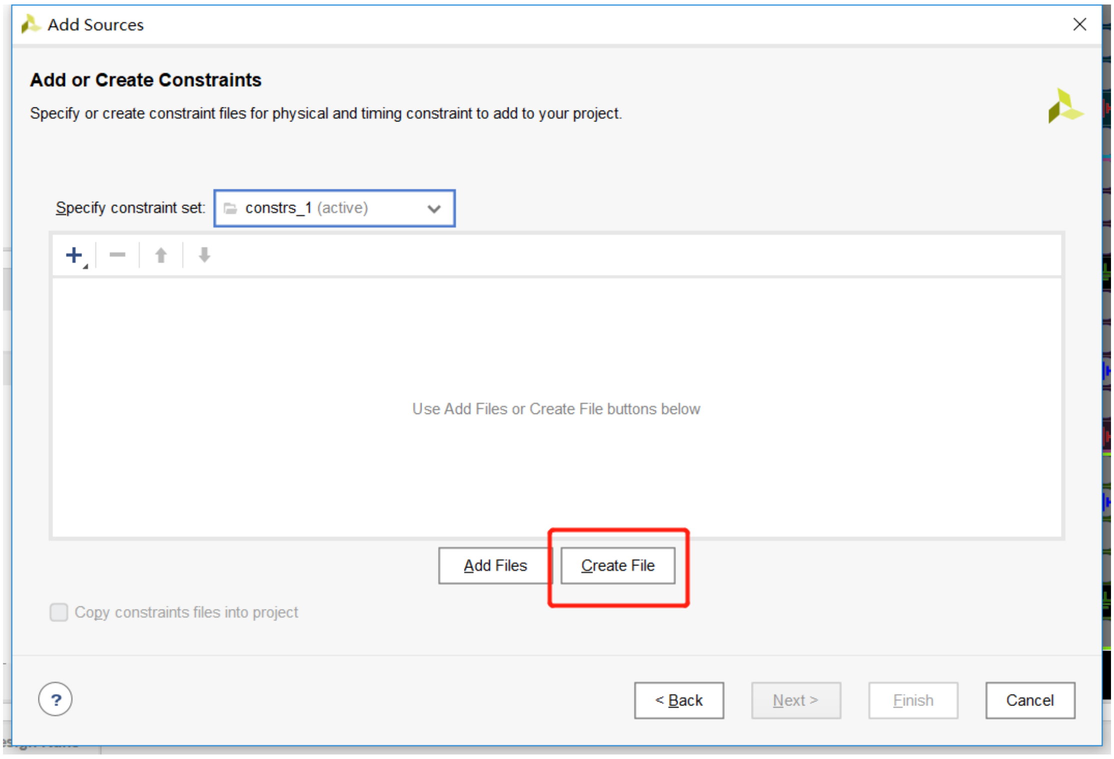
Click “**Add Sources**”



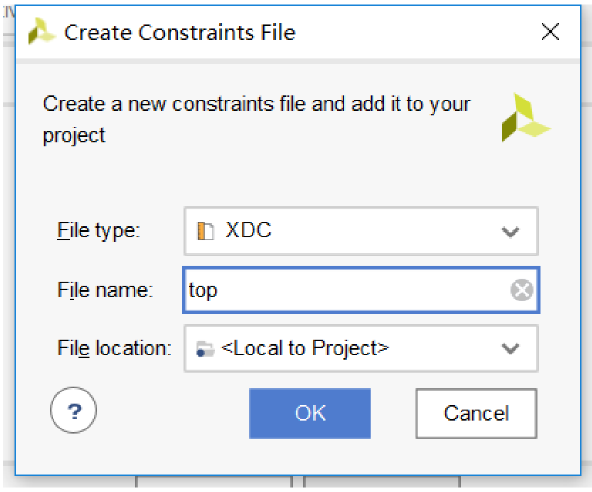
Select “**Add or create constraints**”, and click “**Next**”



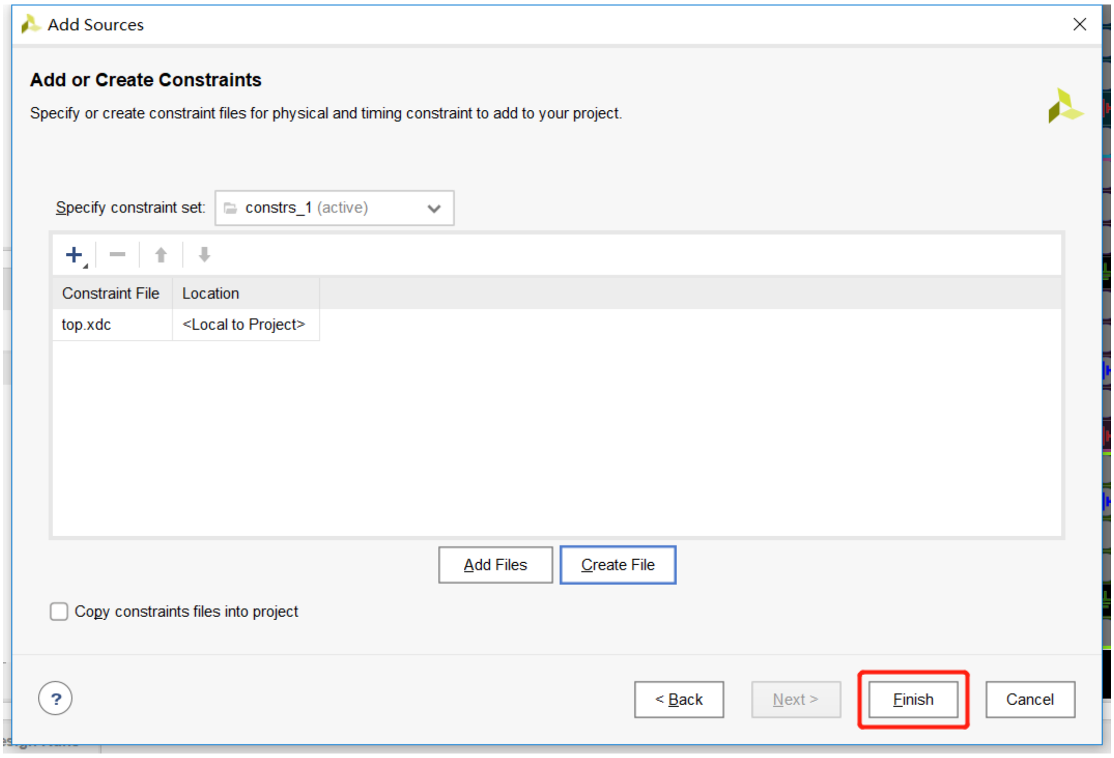
Click “**Create File**”



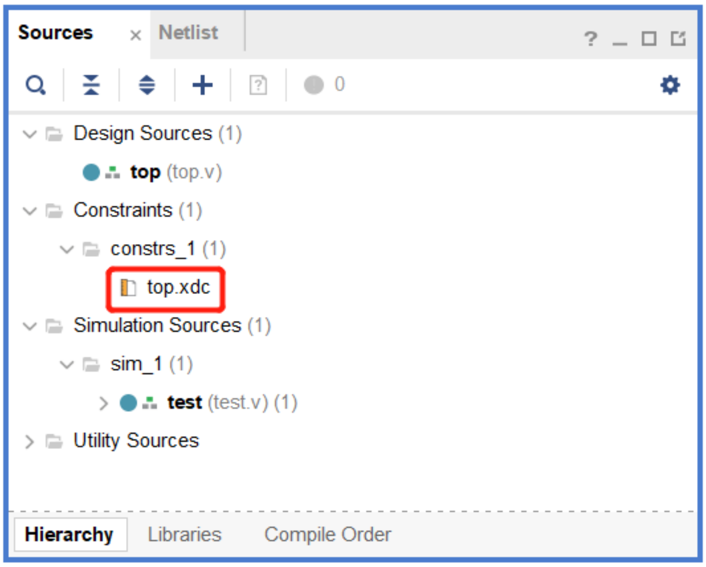
Write the File name, then click “**OK**”



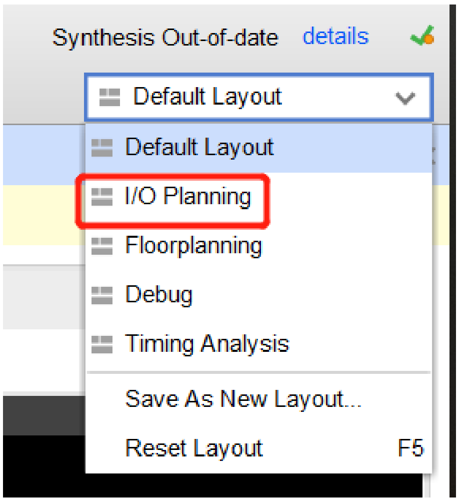
Click "**Finish**"



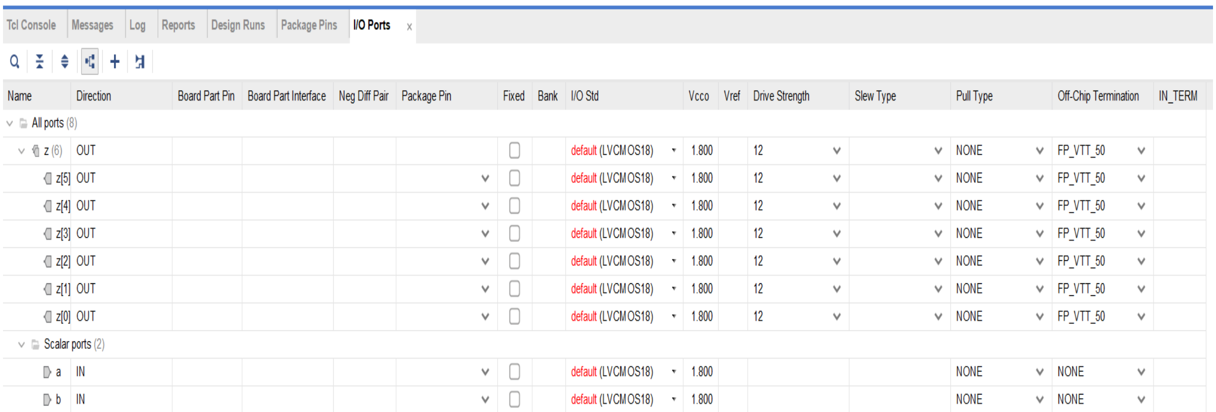
Then we have added Constraints file: **top.xdc**



Then we select "**I/O Planning**" at top right corner.



Next, we will assign Pins,



By Zedborad's Hardware User's guide

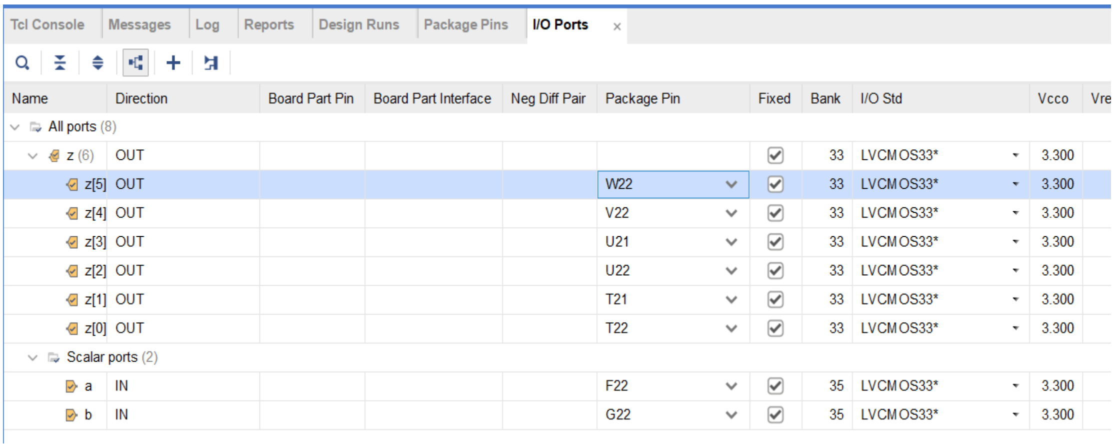
Table 13 - DIP Switch Connections

Signal Name	Zynq pin
SW0	F22
SW1	G22
SW2	H22
SW3	F21
SW4	H19
SW5	H18
SW6	H17
SW7	M15

Table 14 - LED Connections

Signal Name	Subsection	Zynq pin
LD0	PL	T22
LD1	PL	T21
LD2	PL	U22
LD3	PL	U21
LD4	PL	V22
LD5	PL	W22
LD6	PL	U19
LD7	PL	U14
LD9	PS	D5 (MIO7)

Assign ports

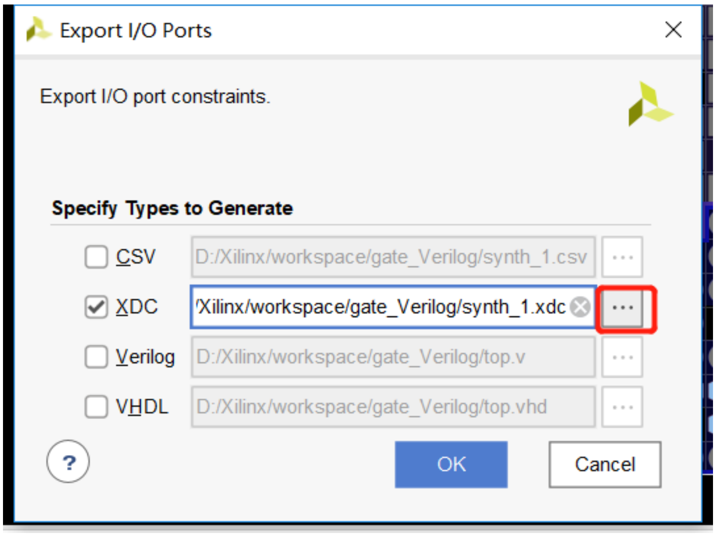


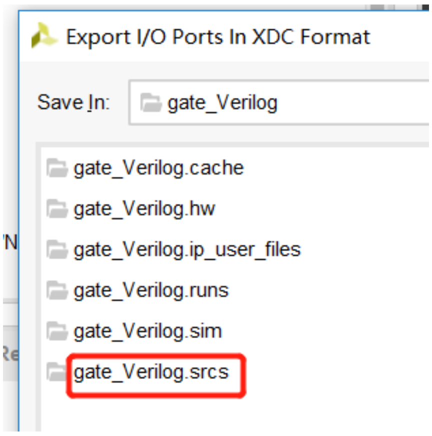
Tcl Console | Messages | Log | Reports | Design Runs | Package Pins | I/O Ports

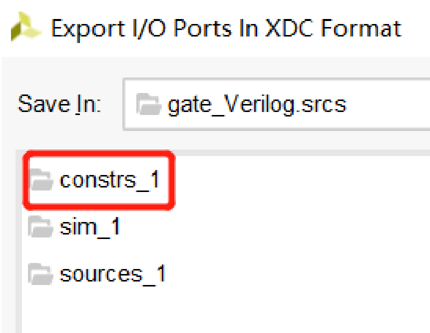
Name	Direction	Board Part Pin	Board Part Interface	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std	Vcco	Vre
All ports (8)										
z (6)	OUT					✓	33	LVCMOS33*	3.300	
z[5]	OUT				W22	✓	33	LVCMOS33*	3.300	
z[4]	OUT				V22	✓	33	LVCMOS33*	3.300	
z[3]	OUT				U21	✓	33	LVCMOS33*	3.300	
z[2]	OUT				U22	✓	33	LVCMOS33*	3.300	
z[1]	OUT				T21	✓	33	LVCMOS33*	3.300	
z[0]	OUT				T22	✓	33	LVCMOS33*	3.300	
Scalar ports (2)										
a	IN				F22	✓	35	LVCMOS33*	3.300	
b	IN				G22	✓	35	LVCMOS33*	3.300	

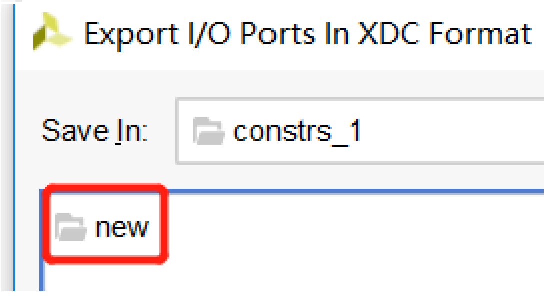
port	pin	I/O standard
a	F22	LVCMOS33
b	G22	LVCMOS33
z[0]	T22	LVCMOS33
z[1]	T21	LVCMOS33
z[2]	U22	LVCMOS33
z[3]	U21	LVCMOS33
z[4]	V22	LVCMOS33
z[5]	W22	LVCMOS33

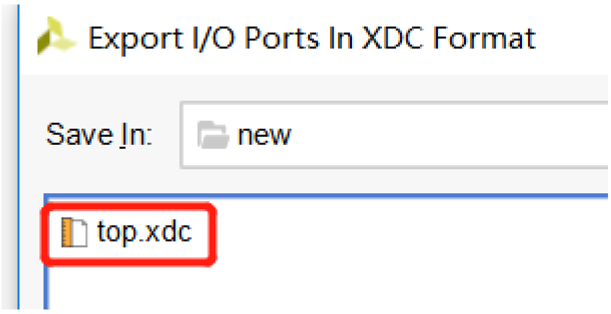
Right click, select "**Export I/O Ports**" in the figure above. **Select XDC**, don't select CSV, and path points to current project "**top.xdc**""



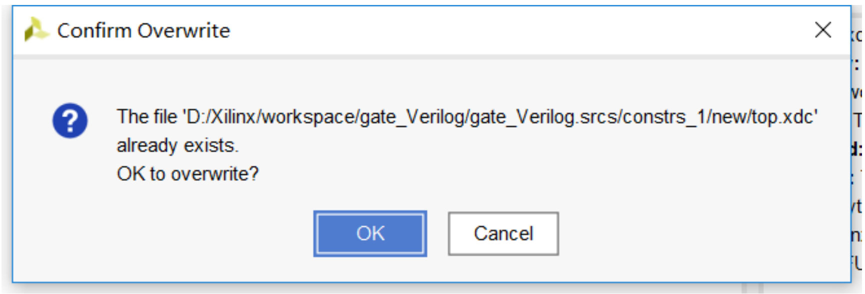




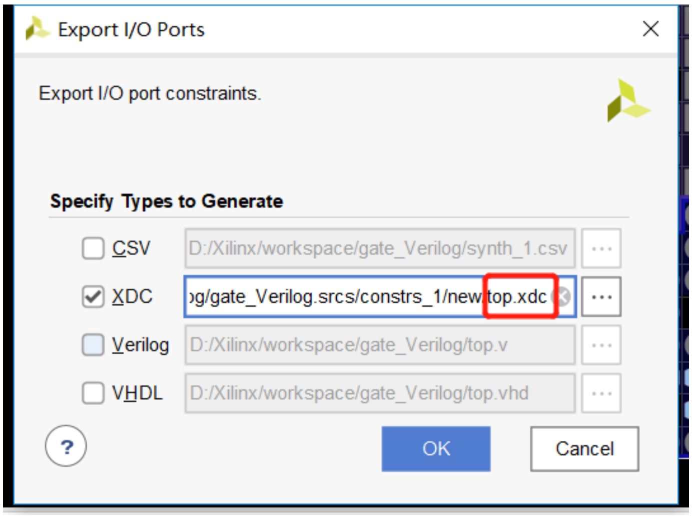




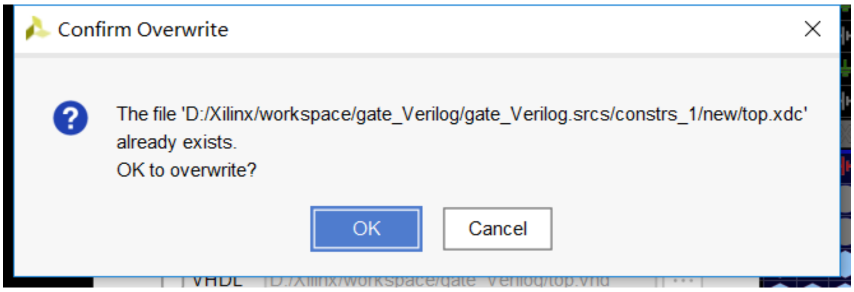
Double click “**top.xdc**”,then click “**Ok**”



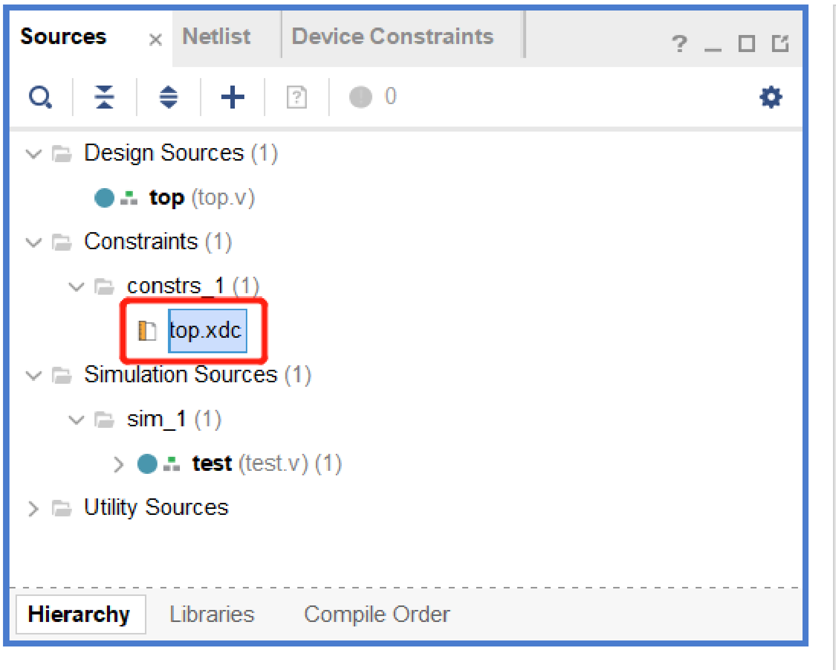
Click “**OK**”

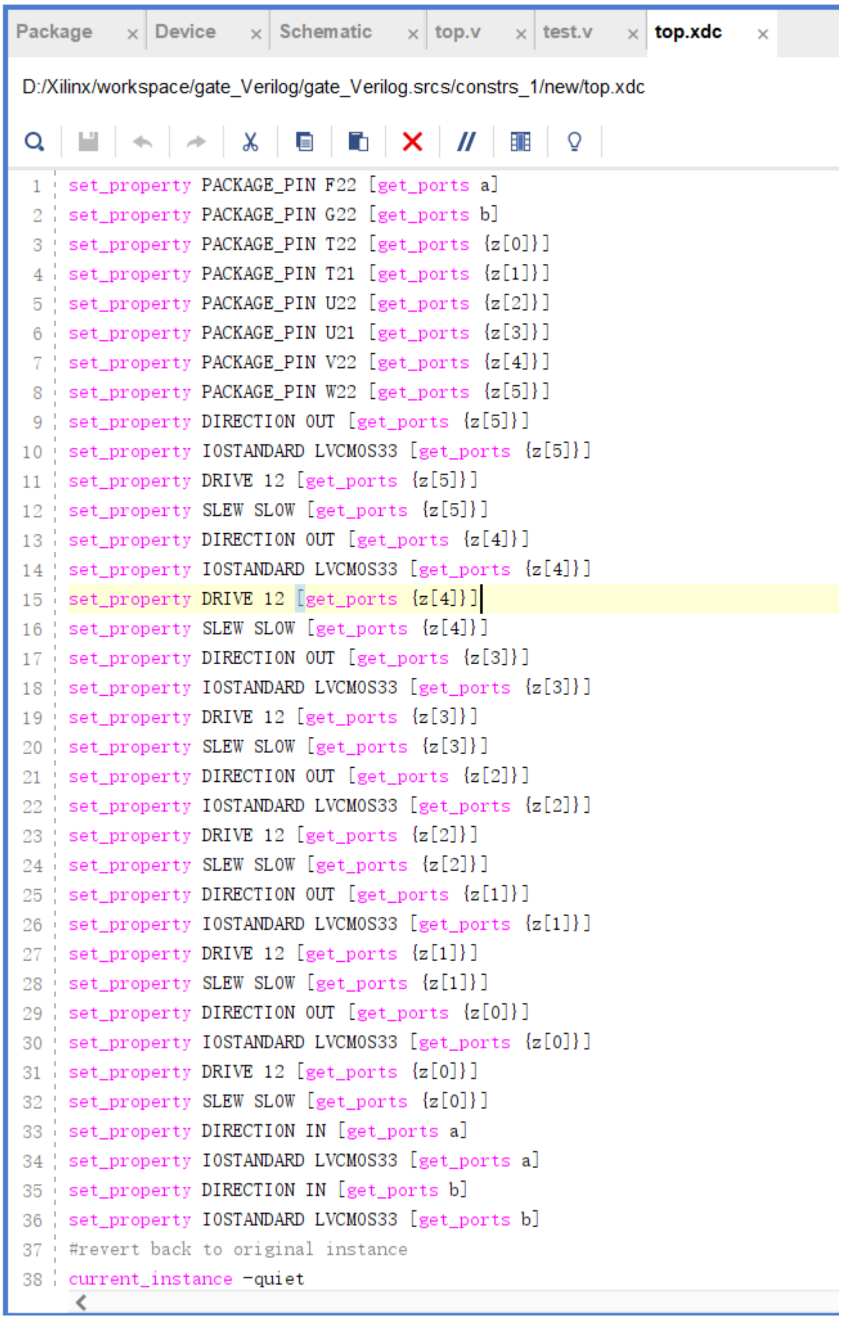


Click "**OK**"



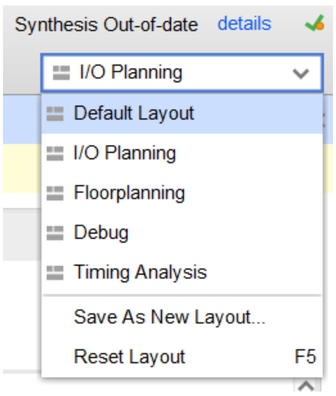
Find and open "**top.xdc**".and we can check constraints code.





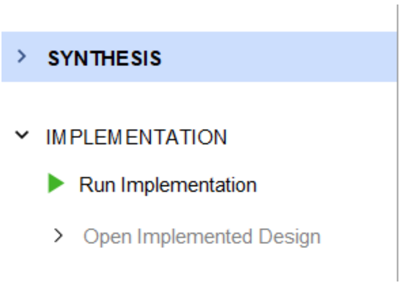

```
set_property PACKAGE_PIN F22 [get_ports a]
set_property PACKAGE_PIN G22 [get_ports b]
set_property PACKAGE_PIN T22 [get_ports {z[0]}]
set_property PACKAGE_PIN T21 [get_ports {z[1]}]
set_property PACKAGE_PIN U22 [get_ports {z[2]}]
set_property PACKAGE_PIN U21 [get_ports {z[3]}]
set_property PACKAGE_PIN V22 [get_ports {z[4]}]
set_property PACKAGE_PIN W22 [get_ports {z[5]}]
set_property DIRECTION OUT [get_ports {z[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {z[5]}]
set_property DRIVE 12 [get_ports {z[5]}]
set_property SLEW SLOW [get_ports {z[5]}]
set_property DIRECTION OUT [get_ports {z[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {z[4]}]
set_property DRIVE 12 [get_ports {z[4]}]
set_property SLEW SLOW [get_ports {z[4]}]
set_property DIRECTION OUT [get_ports {z[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {z[3]}]
set_property DRIVE 12 [get_ports {z[3]}]
set_property SLEW SLOW [get_ports {z[3]}]
set_property DIRECTION OUT [get_ports {z[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {z[2]}]
set_property DRIVE 12 [get_ports {z[2]}]
set_property SLEW SLOW [get_ports {z[2]}]
set_property DIRECTION OUT [get_ports {z[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {z[1]}]
set_property DRIVE 12 [get_ports {z[1]}]
set_property SLEW SLOW [get_ports {z[1]}]
set_property DIRECTION OUT [get_ports {z[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {z[0]}]
set_property DRIVE 12 [get_ports {z[0]}]
set_property SLEW SLOW [get_ports {z[0]}]
set_property DIRECTION IN [get_ports a]
set_property IOSTANDARD LVCMOS33 [get_ports a]
set_property DIRECTION IN [get_ports b]
set_property IOSTANDARD LVCMOS33 [get_ports b]
#revert back to original instance
current_instance -quiet
```

Return to “**Default Layout**” at right top corner.

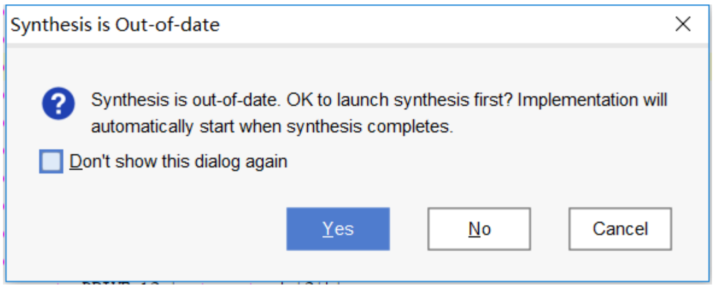


Step7: Implementation

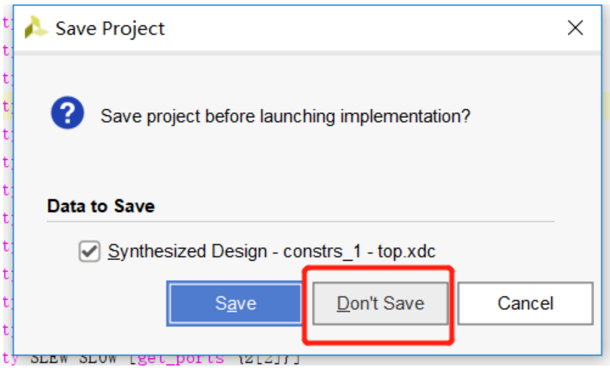
Click "**Run Implementation**"



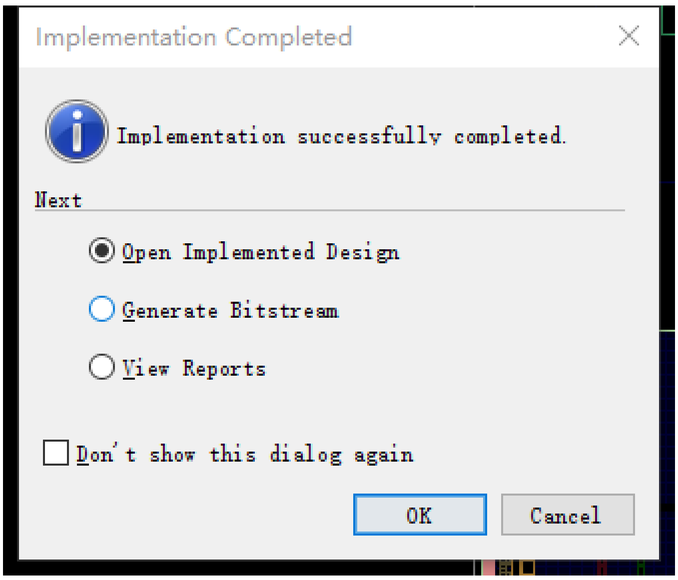
Click "**Yes**"



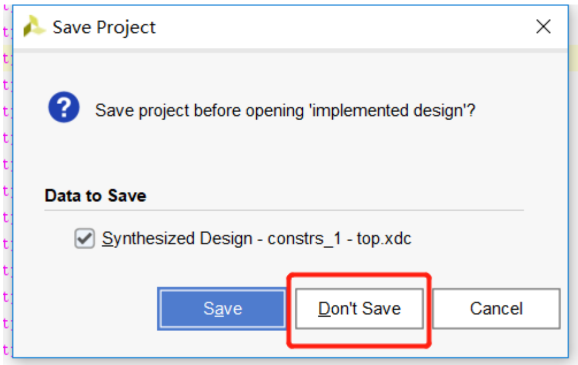
Click "**Don't save**"



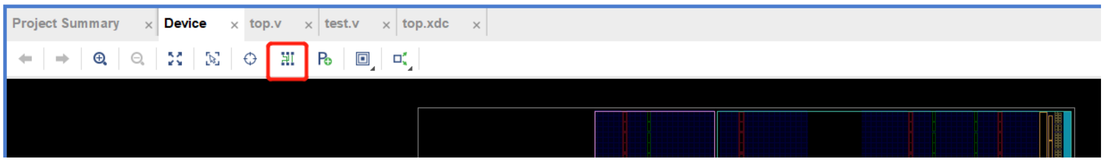
After Implementation, Click "**OK**" and open Implemented Design



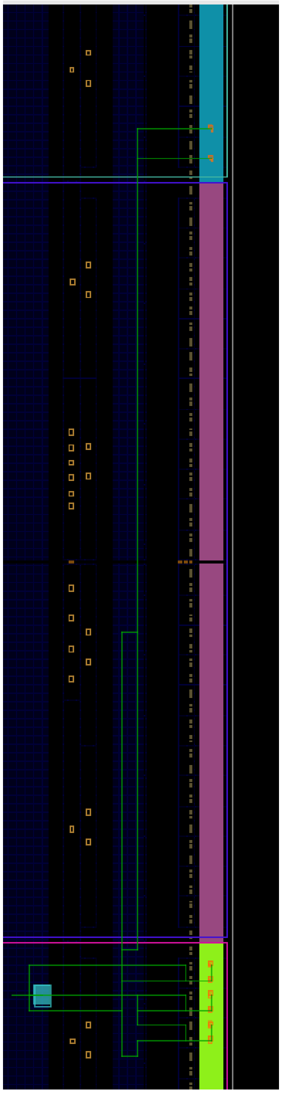
Click “**Don’t save**”

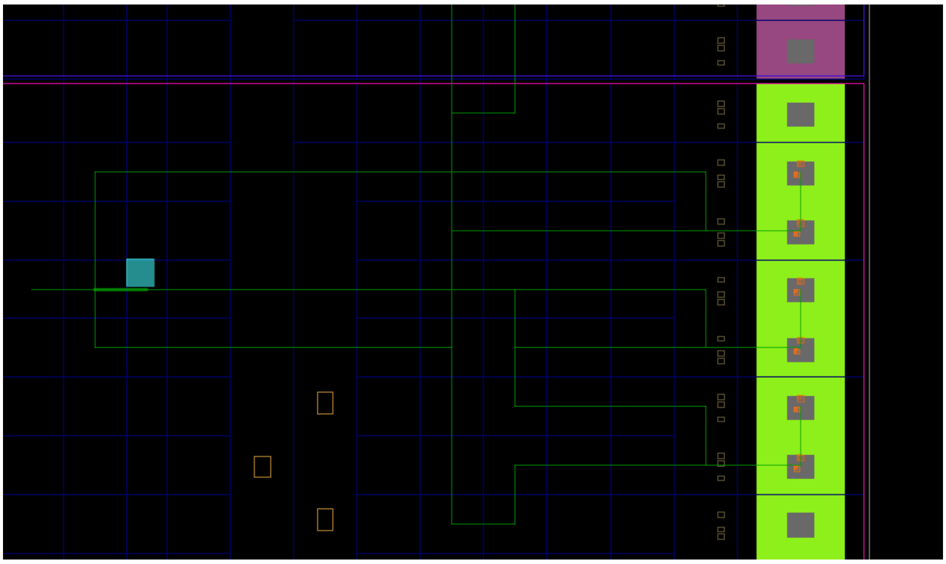


Click “**Routing Resources**”



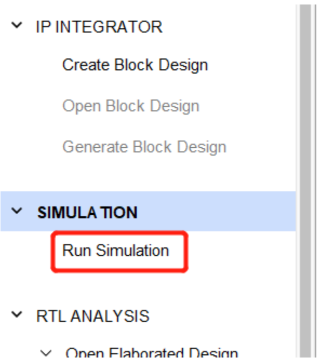
Click  under Device graph, adjust the suitable location. check routing



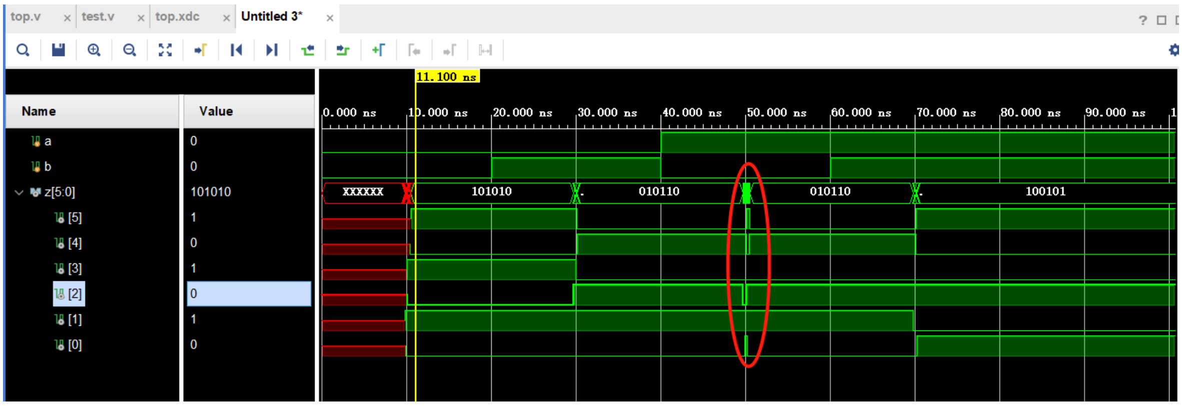


Step8: Run Post-Implementation Timing Simulation

Click "**Run Simulation**", select "**Run Post-Implementation Timing Simulation**", Then select Don't save.

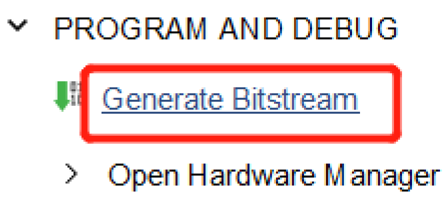


We can see that there is a little delay from simulation.

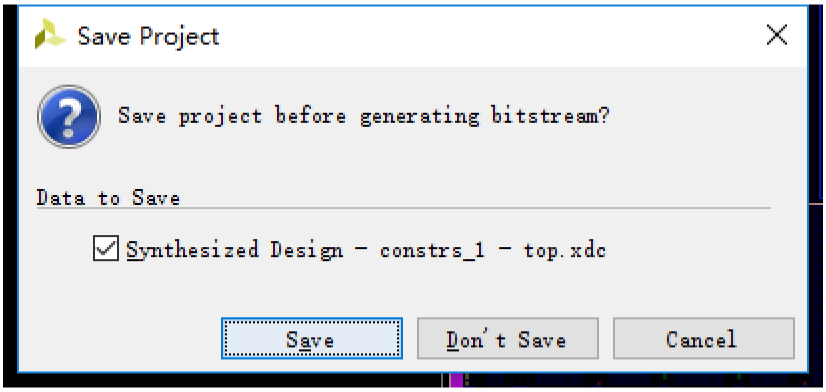


Step9: Generate Bitstream

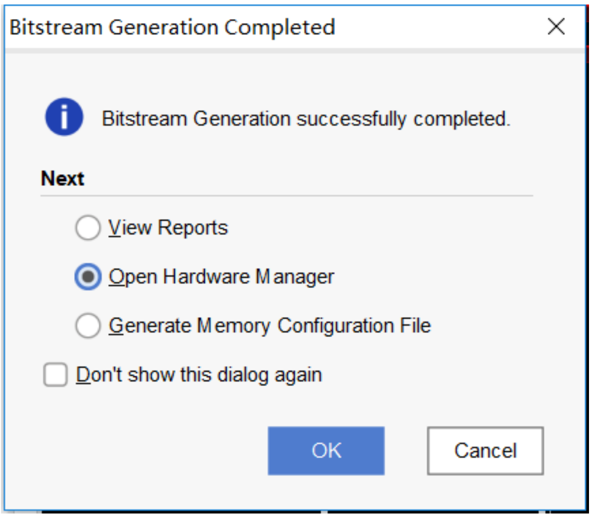
Click "**Generate Bitstream**"



Click “**Don’t Save**”

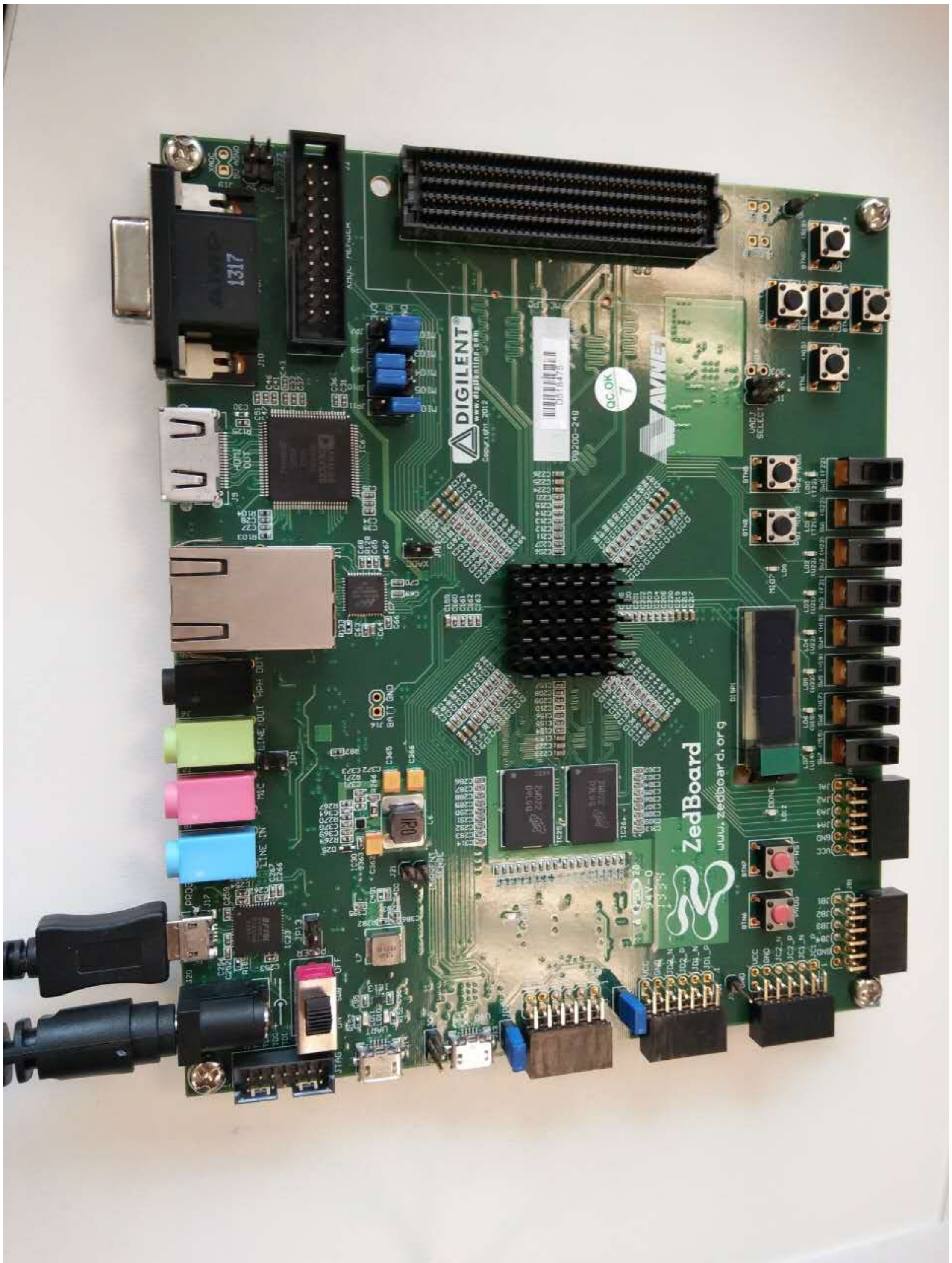


Select “**Open Hardware Manager**”, and click “**OK**”

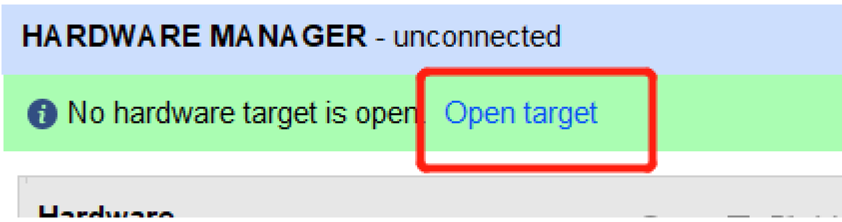


Step10: Connect Zedboard and Configure FPGA,

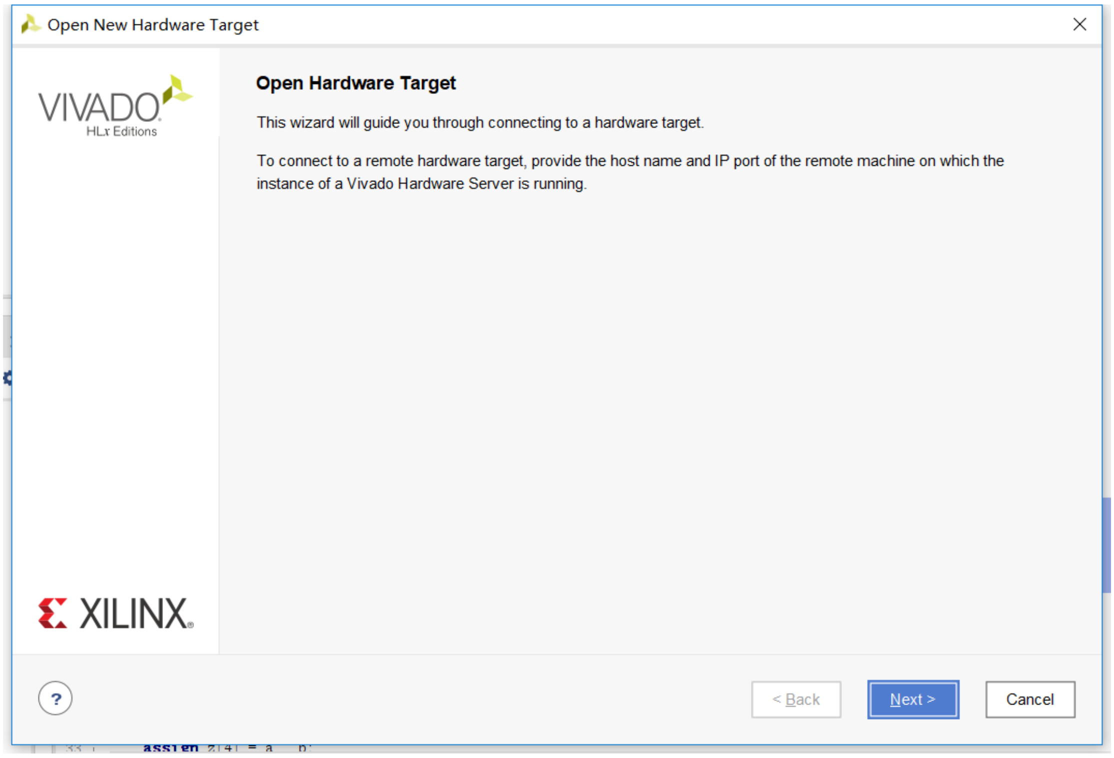
Link power and download cable through a Micro B USB connector, **J17**.and open power switch.



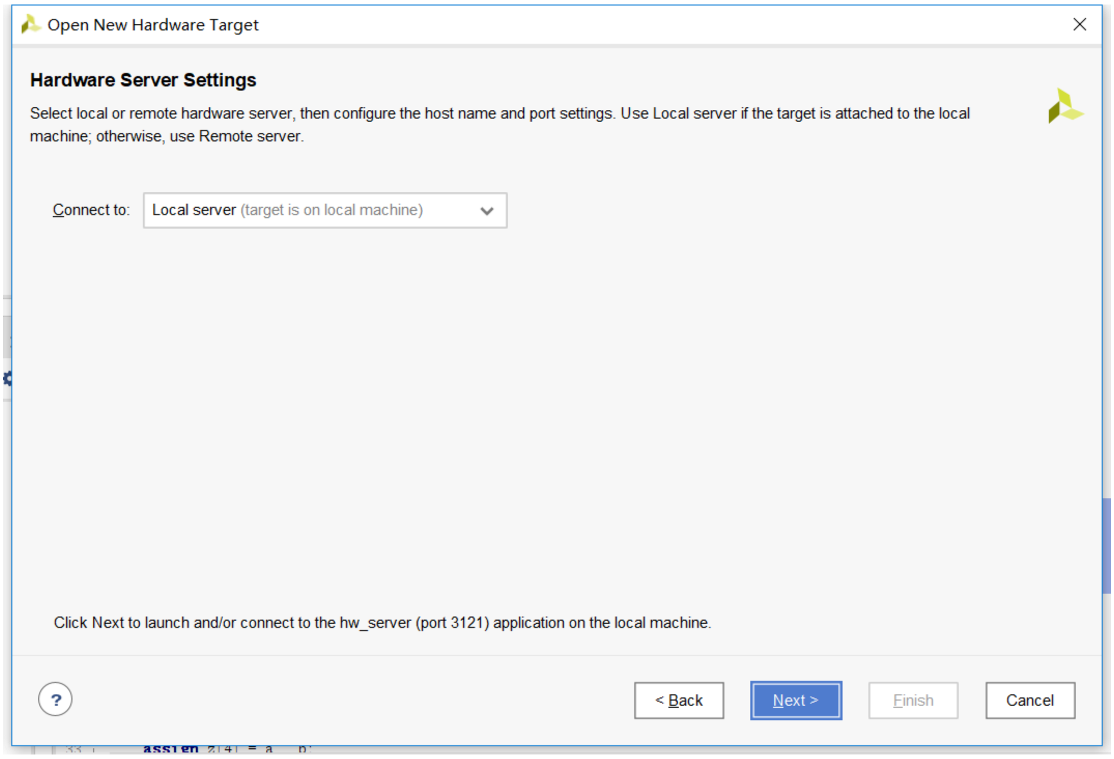
Click “**Open target**” and select “**Open New Target**”



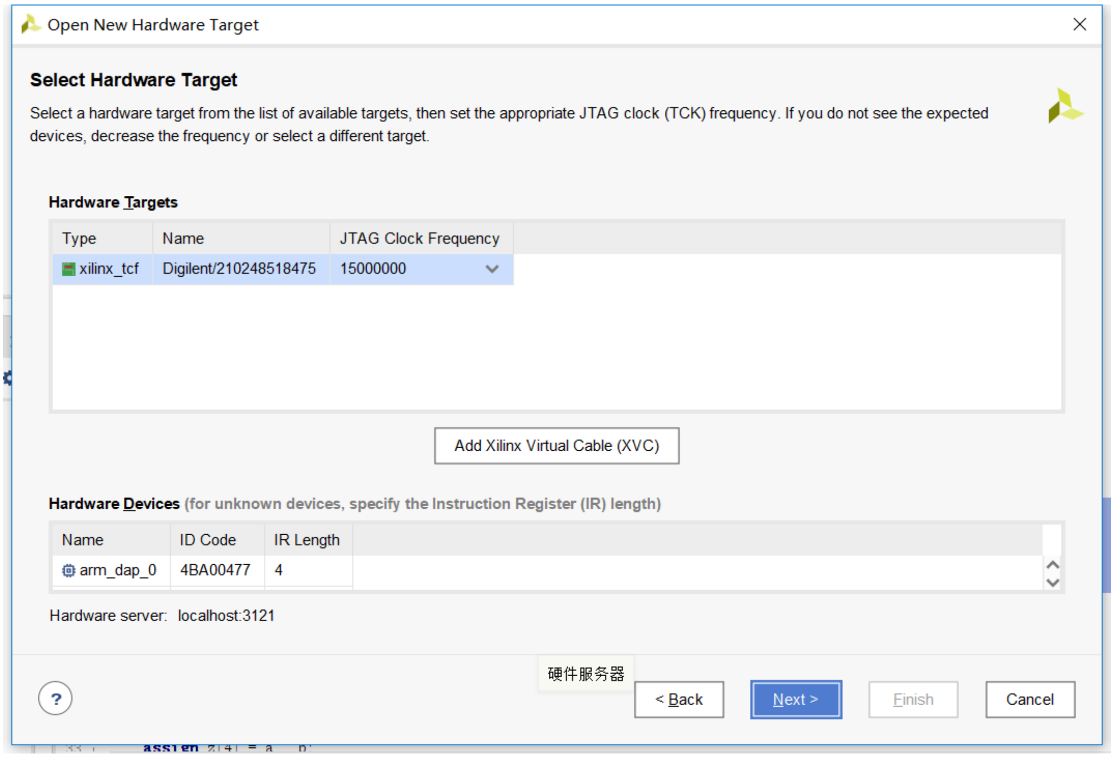
Click “Next”



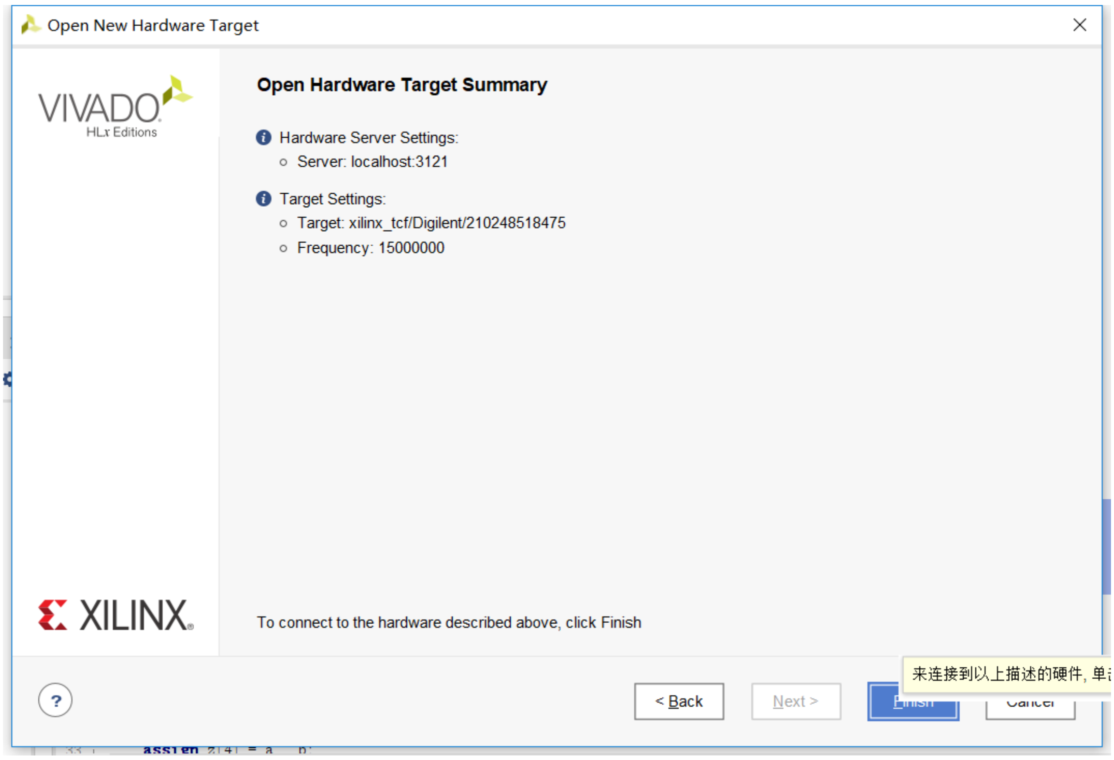
Click “**Next**”



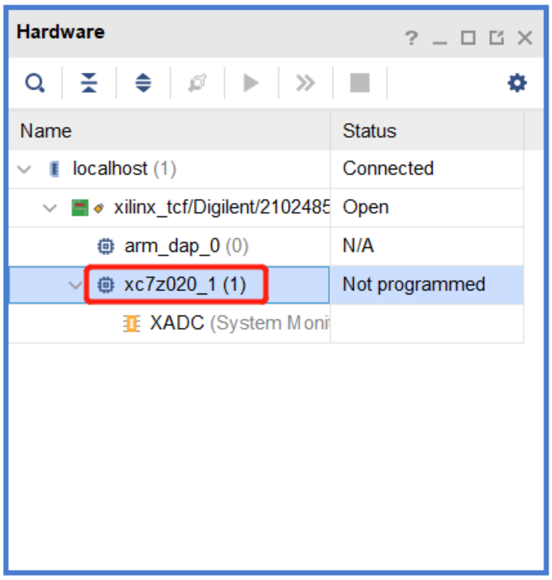
Click “**Next**”



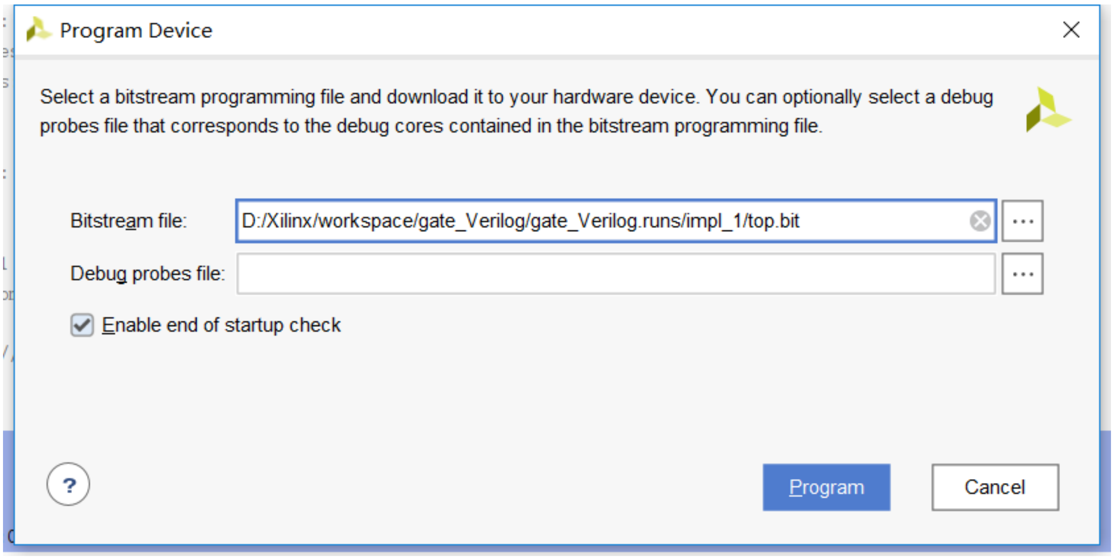
Click “**Finish**”



Right click device “**xc7z020_1**”, select “**Program Device**”



Click “**Program**”

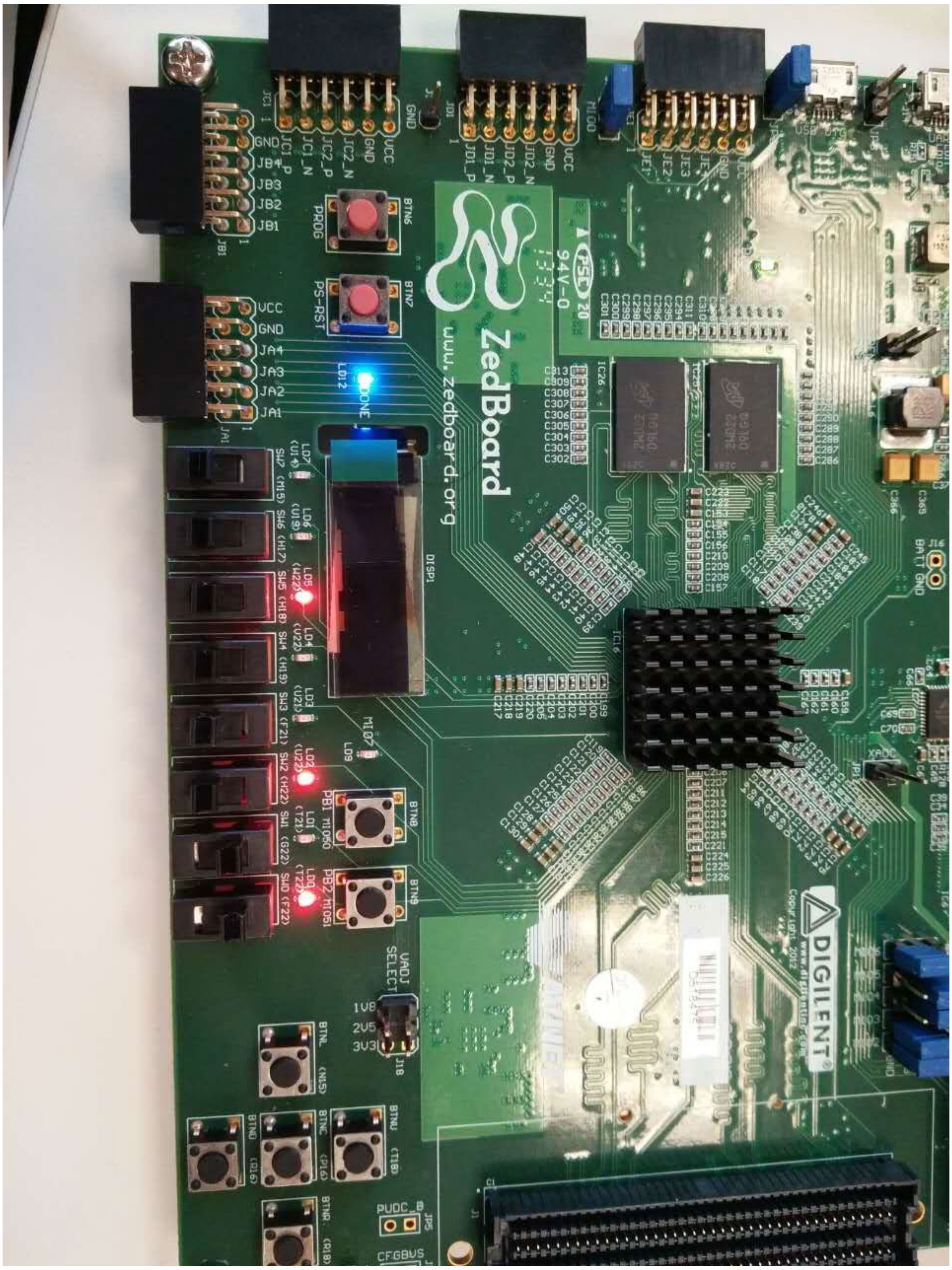


Verify the correctness by the switch sw1,sw0 through the board, compared to the truth table

.

Truth Table

a	b	Z[5:0]
0	0	101010
0	1	010110
1	0	010110
1	1	100101



Exercise:

According to previous memory、clock divider and address counter lab, now you need to integrate this lab and download to Zedboard using upper tutorial. Here we need clock source from pin Y9 on the basis of manual.

2.5 Clock sources

The Zynq-7000 AP SoC's PS subsystem uses a dedicated 33.3333 MHz clock source, IC18, Fox 767-33.333333-12, with series termination. The PS infrastructure can generate up to four PLL-based clocks for the PL system. An on-board 100 MHz oscillator, IC17, Fox 767-100-136, supplies the PL subsystem clock input on bank 13, pin Y9.